
中国矿业大学计算机学院

2023 级本科生课程实验报告

课程名称: 大数据分析实践

报告时间: 2026. 6. 20

学生姓名: 王志豪

学 号: 08232337

专业班级: 数据科学与大数据技术 1 班

任课教师: 唐朝刚

成绩考核

编号	课程教学目标	占比	得分
1	目标 1：具备扎实的大数据工程基础知识，了解解决大数据复杂工程问题的基本方法，并遵循复杂系统开发的工程化基本要求；	20%	
2	目标 2：能够在大数据系统中合理地组织、存储、处理、分析和应用数据，正确地设计算法并对算法进行分析和评价；	20%	
3	目标 2：能够基于大数据基本原理并采用科学方法对大数据软硬件系统设计合理的实验方案。	60%	
总成绩			
指导教师		评阅日期	

【自动插入目录（通过菜单 引用/目录）】

【文档中的格式，必须使用工具栏中的样式】

【正文图表要求有图表标题，且自动编号】

【打开 视图/导航窗格，必须能够看到清晰的文档结构】

目录

- 实验一 贝叶斯分类.....1
 - 1. 实验基本信息.....1
 - 2. 实验目标和实验环境.....1
 - 2.1 实验目标.....1
 - 2.2 实验环境.....2
 - 3. 实验内容.....2
 - 3.1 算法原理.....2
 - 3.2 特征选择.....3
 - 3.3 对照实验设计.....5
 - 4. 实验步骤和结果.....5
 - 4.1 数据预处理.....5
 - 4.2 标签编码与数据集划分.....7
 - 4.3 模型训练.....8
 - 4.4 实验结果.....8
 - 5. 结论与讨论.....14
 - 5.1 实验结论.....14
 - 5.2 朴素贝叶斯特征的独立性假设分析.....15
 - 5.3 实验心得.....15
- 实验二 K-Means 算法.....17
 - 1. 实验基本信息.....17
 - 2. 实验目标和实验环境.....17
 - 2.1 实验目标.....17
 - 2.1 实验环境.....18
 - 3. 实验内容.....18
 - 3.1 算法原理.....18
 - 3.2 特征选择.....19
 - 4. 实验步骤和结果.....20
 - 4.1 数据预处理.....20
 - 4.2 特征标准化.....22
 - 4.3 K 值选择.....23
 - 4.3 PCA 降维分析.....25
 - 4.4 聚类结果.....26
 - 5. 结论与讨论.....29
 - 5.1 实验结论.....29
 - 5.2 KMeans 低分原因总结.....30
 - 5.3 实验心得.....30
- 实验三 ID3 分类算法.....31
 - 1. 实验基本信息.....31
 - 2. 实验目标和实验环境.....31
 - 2.1 实验目标.....31
 - 2.2 实验环境.....32

3.	实验内容	33
3.1	算法原理	33
4.	实验步骤和结果	34
4.1	数据预处理	34
4.2	特征离散化	34
4.3	ID3 算法核心实现	35
4.4	实验结果	39
4.5	3 分类结果	41
5.	结论与讨论	43
实验四	C4.5 分类算法	45
1.	实验基本信息	45
2.	实验目标和实验环境	45
2.1	实验目标	45
2.2	实验环境	46
3.	实验内容	46
3.1	算法原理	46
3.2	特征工程	48
4.	实验步骤和结果	49
4.1	数据预处理	49
4.2	ID3 算法核心实现	49
4.3	实验结果	54
4.4	实验优化	56
5.	结论与讨论	59
5.1	实验结论	59
实验五	深度神经网络算法应用	60
1.	实验基本信息	60
2.	实验目标和实验环境	61
2.1	实验目标	61
2.2	实验环境	61
3.	实验内容	62
3.1	算法原理	62
3.2	特征选择	64
4.	实验步骤和结果	66
4.1	数据预处理	66
4.2	模型训练	70
4.3	实验结果	72
5.	结论与讨论	78
5.1	实验结论	78
5.2	实验心得	80

实验一 贝叶斯分类

1. 实验基本信息

实验名称：基于朴素贝叶斯的 NBA 球员场上位置预测

实验日期：2026 年 6 月 15 日

实验地点：计算机楼

2. 实验目标和实验环境

2.1 实验目标

本实验旨在利用朴素贝叶斯 (Naive Bayes) 分类算法，根据 NBA 球员的历史技术统计特征，预测其场上位置。实验设两个子任务：

- (1). **5 位置细粒度分类**：预测球员的具体位置 (C 中锋、PF 大前锋、SF 小前锋、SG 得分后卫、PG 控球后卫)；
- (2). **3 超类粗粒度分类**：将混淆严重的相邻位置合并为 Big (C+PF)、Wing (SF+SG)、PG 三个超类，验证标签粒度对分类性能的影响。

预处理阶段引入 per-36 分钟归一化和 TOT 去重等改进，以消除上场时间和数据冗余带来的噪声。

2.2 实验环境

表 1-1 实验环境

项目	配置
硬件	CPU: Intel Core i7, 16GB RAM
操作系统	(1). Windows 11
编程语言	Python 3.x
核心库	pandas, numpy, scikit-learn, matplotlib, seaborn
分类算法	GaussianNB
评估指标	Accuracy, Precision, Recall, F1-score, Confusion Matrix

3. 实验内容

3.1 算法原理

朴素贝叶斯是一种基于**贝叶斯定理**的概率分类器，其核心假设是特征之间**条件独立**（朴素假设）。对于给定的输入

样本 $x = (x_1, x_2, \dots, x_n)$ ，算法计算其属于类别 C_k 的后验概率：

$$P(C_k | x) = \frac{P(C_k) \prod_{i=1}^n P(x_i | C_k)}{P(x)}$$

并选择后验概率最大的类别作为预测结果。

本实验采用 **GaussianNB**，它假设每个类别中连续特征的似然 $P(x_i | C_k)$ 服从高斯（正态）分布：

$$P(x_i | C_k) = \frac{1}{\sqrt{2\pi\sigma_{k,i}^2}} \exp \left(-\frac{(x_i - \mu_{k,i})^2}{2\sigma_{k,i}^2} \right)$$

其中 $\mu_{k,i}$ 和 $\sigma_{k,i}^2$ 分别是类别 C_k 中第 i 个特征的均值和方差，由训练数据估计得到。

3.2 特征选择

本实验选取 **12 维特征**，相比原实验的 8 维场均特征，进行了以下改进：

- **per-36 分钟归一化**替代场均化，消除上场时间差异；
- **eFG%** 替代 FG%，引入三分加权；
- 新增**出手密度特征**（3PA_per36、FTA_per36）和**效率特征**（TOV_per36、PF_per36）。

表 2-2 输入特征

编号	特征	含义	选取理由
1	PTS_per36	每 36 分钟得分	得分能力，per-36 消除上场时间差异
2	TRB_per36	每 36 分钟篮板	中锋/大前锋篮板显著高于后卫
3	AST_per36	每 36 分钟助攻	控球后卫助攻显著高于其他位置
4	STL_per36	每 36 分钟抢断	外线球员抢断更多
5	BLK_per36	每 36 分钟盖帽	中锋盖帽远高于其他位置
6	3P%	三分命中率	外线球员三分更好
7	FT%	罚球命中率	后卫罚球更准
8	eFG%	有效命中率	含三分加权，比 FG% 更反映真实投篮效率
9	3PA_per36	每 36 分钟三分出手	三分出手倾向，区分"不投"和"投不进"
10	FTA_per36	每 36 分钟罚球出手	造罚球倾向，内线球员通常更高
11	TOV_per36	每 36 分钟失误	持球多的球员失误更高
12	PF_per36	每 36 分钟犯规	内线球员犯规偏多

3.3 对照实验设计

表 3-3 对照试验设计

实验组	特征维度	特征集	目的
5 位置分类	12 维	上述 12 维	基线细粒度分类
3 超类分类	12 维	同上，标签合并为 Big/Wing/PG	验证粗粒度分类性能提升

4. 实验步骤和结果

4.1 数据预处理

- 衍生 per-36 指标

原始数据中所有技术统计为赛季累计值（PTS、TRB、AST 等），直接使用会同时受出场次数 G 和上场时间 MP 的影响。本实验采用 **per-36 分钟归一化**替代原先的场均化：

```
# Per-36 衍生字段核心代码
```

```
per36_map = {
    'PTS_per36': 'PTS', 'TRB_per36': 'TRB', 'AST_per36': 'AST',
    'STL_per36': 'STL', 'BLK_per36': 'BLK', 'PF_per36': 'PF',
    'TOV_per36': 'TOV', '3PA_per36': '3PA', 'FTA_per36': 'FTA',
}
for new_f, raw_f in per36_map.items():
    df[new_f] = df[raw_f] / df['MP'].replace(0, np.nan) * 36
```

- eFG%补充

```
df['eFG%'] = df['eFG%'].fillna((df['FG'] + 0.5 * df['3P']) / df['FGA'])
```

eFG%（有效命中率）给三分球赋予 1.5 倍权重，更准确反映真实投篮效率。数据集中部分 eFG% 已存在，未计算的行在此补充。

● 缺失值填充

```
for col in ['FG%', '3P%', 'FT%', '2P%', 'eFG%']:
    df[col] = df[col].fillna(0)
```

缺失情况：3P% 缺失最多（3,485 条），集中在三分未普及的早期赛季。所有缺失值均填充为 0，因为缺失原因均为对应出手数为 0。

● TOT 去重

```
1980,Tim Bassett,PF,28,TOT,12,164,... ← 合计行（12 场）
1980,Tim Bassett,PF,28,NJN,7,92,... ← 仅在 NJN（7 场）
1980,Tim Bassett,PF,28,SAS,5,72,... ← 仅在 SAS（5 场）
```

若直接剔除 TOT 而保留单球队行，同一个球员同一年会多次出现，随机划分时可能导致**数据泄露**（同一人的不同分片同时进入训练集和测试集）。

```
# TOT 去重核心代码
# 每个 (Year, Player) 组若有 TOT 行，只保留 TOT，删单球队行
has_tot = df.groupby(['Year', 'Player'])['Tm'].transform(
    lambda x: (x == 'TOT').any()
)
df = df[~(has_tot & (df['Tm'] != 'TOT'))].copy()
```

● 行过滤

```
df = df[df['MP'] >= 200] # 上场时间足够长，保证 per-36 可靠
df = df[df['G'] >= 10] # 出场次数足够多
```

相比原始实验的 $G \geq 20$ ，改用 $MP \geq 200$ 更为精确。一个 $G=82$ 场均 2.5 分钟的球员 $MP \approx 205$ ，其 per-36 数

据虽然通胀但样本量尚可；而 $G=20$ 场均 5 分钟的球员 $MP \approx 100$, per-36 的可靠性较差。 $MP \geq 200$ 是更合理的底线。

4.2 标签编码与数据集划分

● 标签编码

```
# 5 分类标签编码
le5 = LabelEncoder()
le5.fit(['C', 'PF', 'SF', 'SG', 'PG'])
y5 = le5.transform(df['Pos'])

# 3 超类映射
POS_TO_SUPER = {'C': 'Big', 'PF': 'Big', 'PG': 'PG', 'SF': 'Wing', 'SG': 'Wing'}
df['SuperPos'] = df['Pos'].map(POS_TO_SUPER)
le3 = LabelEncoder()
le3.fit(['Big', 'Wing', 'PG'])
y3 = le3.transform(df['SuperPos'])
```

使用 LabelEncoder 将字符串位置标签映射为整数编码。5 分类显式固定顺序 $C=0$, $PF=1$, $SF=2$, $SG=3$, $PG=4$, 避免编码混乱。3 超类将 $C+PF$ 归为 Big (内线)、 $SF+SG$ 归为 Wing (侧翼)、PG 单独保留。

● 训练测试集划分

```
# 分层采样划分核心代码
X_wo_train, X_wo_test, y_train, y_test = train_test_split(
    X_wo, y, test_size=0.3, random_state=42, stratify=y
)
```

采用 **7:3 分层采样**, $\text{stratify}=y$ 保证训练集和测试集中各位置的比例与全量数据一致。

4.3 模型训练

```
def train_and_eval(X_tr, X_te, y_tr, y_te, name, label_names):
    model = GaussianNB()
    model.fit(X_tr, y_tr)
    y_pred = model.predict(X_te)
    y_proba = model.predict_proba(X_te)

    acc = accuracy_score(y_te, y_pred)
    report = classification_report(y_te, y_pred, target_names=label_names,
                                   output_dict=True, zero_division=0)
    cm = confusion_matrix(y_te, y_pred)
    return {'model': model, 'y_pred': y_pred, 'y_proba': y_proba,
            'accuracy': acc, 'report': report, 'confusion_matrix': cm}
```

GaussianNB() 无需调参即可直接训练，因为高斯朴素贝叶斯没有超参数需要调整。predict_proba() 输出每个样本属于各类别的概率，可用于分析分类置信度 zero_division=0 防止在罕见类别的评估中除零报错。

4.4 实验结果

● 5 位置分类结果

表 4-4 5 分类结果

位置	Precision	Recall	F1	Support	分析
SF	0.82	0.83	0.83	783	最佳——出乎意料，per-36 显著改善了 SF 的可分性
C	0.63	0.77	0.69	792	良好——篮板和盖帽特征有效

位置	Precision	Recall	F1	Support	分析
PG	0.56	0.64	0.60	792	中等——部分组织型 PG 助攻偏低
PF	0.52	0.40	0.45	831	较差——与 C 混淆
SG	0.51	0.45	0.48	784	较差——与 SF/PG 混淆

总体指标：

- Accuracy: **0.6138**
- Macro F1: **0.6082**
- Weighted F1: **0.6064**
- 3 位置分类结果

表 5-5 3 分类结果

超类	Precision	Recall	F1	Support	分析
Big	0.87	0.87	0.87	1,623	最佳——内线特征高度独特
Wing	0.79	0.87	0.83	783	良好——侧翼可分性大幅提升
PG	0.80	0.76	0.78	1,576	良好——部分 PG 特征仍不够典型

总体指标：

- Accuracy: **0.8265**
- Macro F1: **0.8266**
- Weighted F1: **0.8259**
- 5 类→3 类映射对比

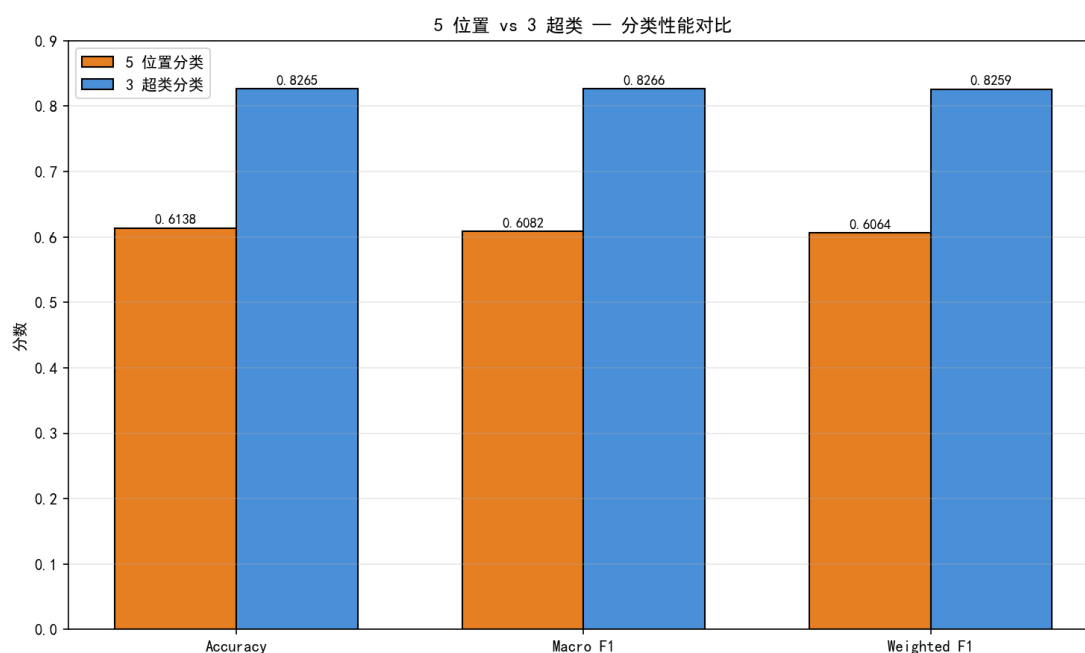


图 1-1 分类性能对比

将 5 分类模型的预测结果映射到 3 超类，与直接 3 超类分类进行比较：

```
# 5 类 -> 3 类映射
y5_pred_super = np.array([POS_TO_SUPER[le5.inverse_transform([p])[0]]
                           for p in res5['y_pred']])
y5_pred_super_enc = le3.transform(y5_pred_super)
acc_map = accuracy_score(y3_test, y5_pred_super_enc)
```

表 6-6 映射结果

指标	5 位置原始	5→3 映射	3 超类直接	直接 vs 映射
Accuracy	0.6138	0.8297	0.8265	-0.0033
Macro F1	0.6082	0.8248	0.8266	+0.0018
Weighted F1	0.6064	0.8262	0.8259	-0.0003

5→3 映射与直接 3 超类的性能几乎完全一致（差距 <0.01）。这说明 5 分类模型的大部分错误发生在**超类内部**（C↔PF、SF↔SG 的混淆），一旦通过映射合并，效果与直接训练 3 超类等价。换言之，GaussianNB 虽然没有在 5 类训练中见过超类标签，但其内部决策边界天然形成了 3 个超类簇。

- 混淆矩阵分析

5 分类混淆矩阵（百分比）：

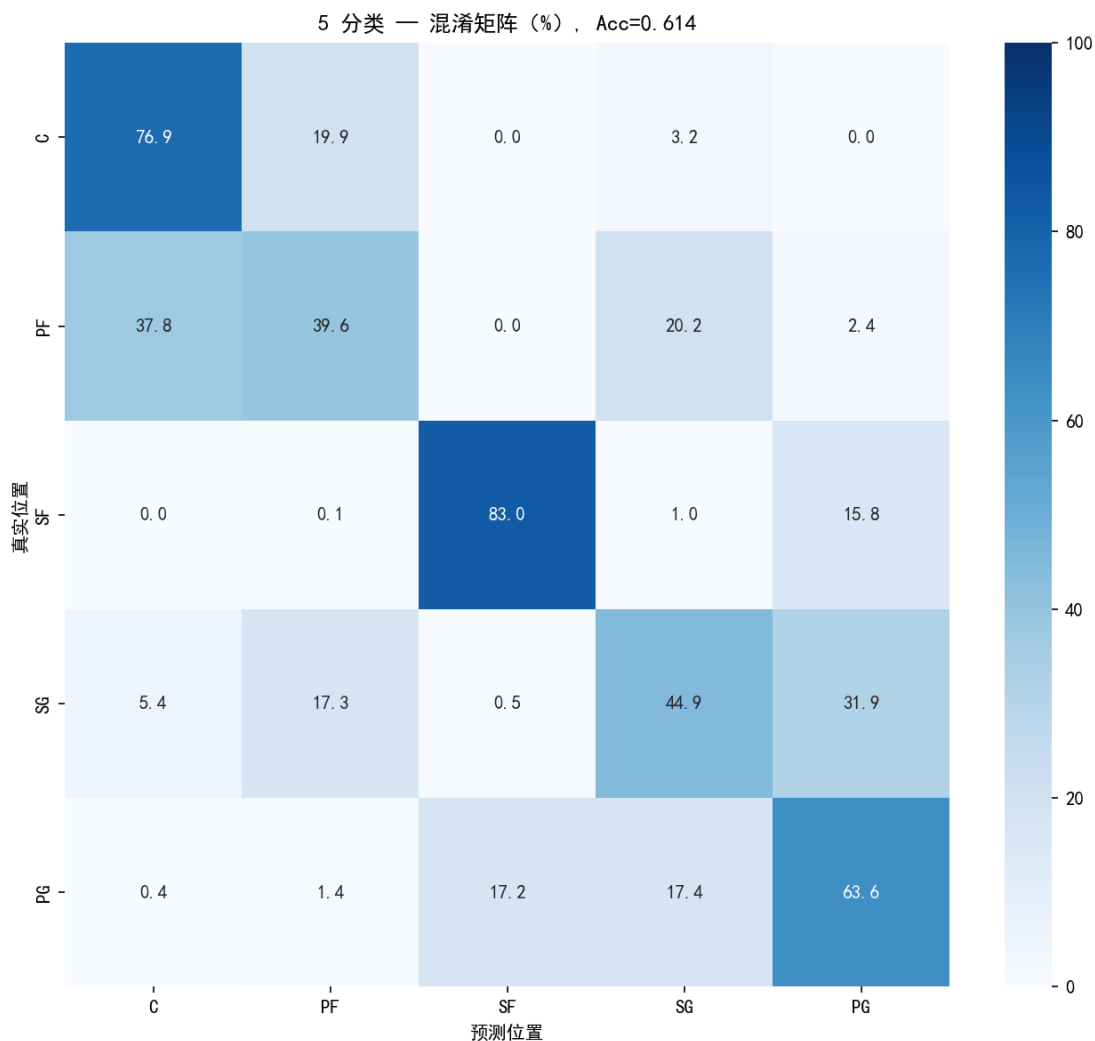


图 2-2 五分类混淆矩阵

(1). 内线 (C 与 PF) 互分严重:

- 真实是 PF 的样本中, 有 314 个被错分为 C (占 PF 总数的 37.8%)。
- 真实是 C 的样本中, 有 158 个被错分为 PF。
- **结论:** 模型很难区分纯中锋和空间型/蓝领大前锋, 这类球员经常摇摆, 导致召回率 (PF 仅为 39.59%) 极低。

(2). 外线 (SG 与 PG) 双向混淆:

- 真实是 **SG** 的样本中，有 **250 个** 被错分为 **PG**，同时有 **136 个** 被错分为 **PF**（可能因为高大型得分后卫被误判）。
- 真实是 **PG** 的样本中，有 **138 个** 被错分为 **SG**，**136 个** 被错分为 **SF**。
- **结论**：双能卫（PG/SG）和侧翼（SG/SF）的界限模糊，这是导致得分后卫（SG）精确率和召回率最低（约 45%~51%）的核心原因。

(3). 小前锋（SF）识别最准：

SF 的精确率和召回率均超过 **82%**，F1 高达 **82.64%**。这说明小前锋的体型和数据特征在模型中具有最高的辨识度，极少被误判为内线（C/PF），被错分的也大多是侧翼（SG/PG）。

3 分类混淆矩阵（百分比）：

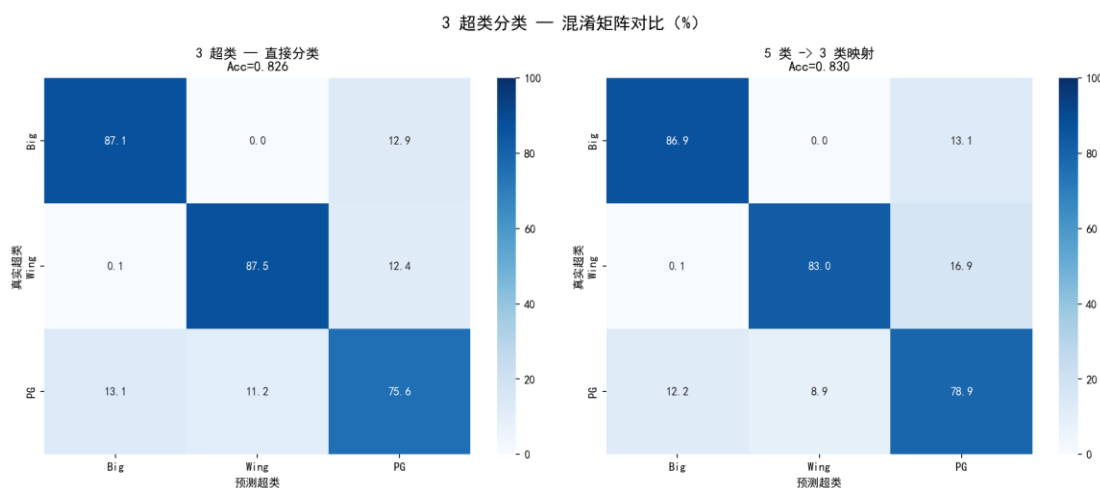


图 3-3 三分类混淆矩阵

Big 和 Wing 的超类纯度高（87%），PG 略低（76%）但

远优于原始实验的 68%。整体分类质量显著高于原始实验。

5. 结论与讨论

5.1 实验结论

(1).**per-36 分钟归一化显著提升了朴素贝叶斯的分类性能。**

5 分类 Accuracy 从 0.537 提升至 **0.614** (+7.6 个百分点), 3 超类 Accuracy 从 0.758 提升至 **0.827** (+6.8 个百分点)。其中 SF 的 Recall 从 31.2% 飙升至 **83.3%**, 说明 per-36 化有效分离了侧翼与其他位置在单位时间效率上的差异。

(2).**5 分类任务中各位置可分性差异显著**: SF (F1=0.83) 和 C (F1=0.69) 最易识别, PF (F1=0.45) 和 SG (F1=0.48) 最易混淆。内线混淆 (C↔PF) 和后场混淆 (SG↔PG) 是分类性能的主要瓶颈。

5.2 朴素贝叶斯特征的独立性假设分析

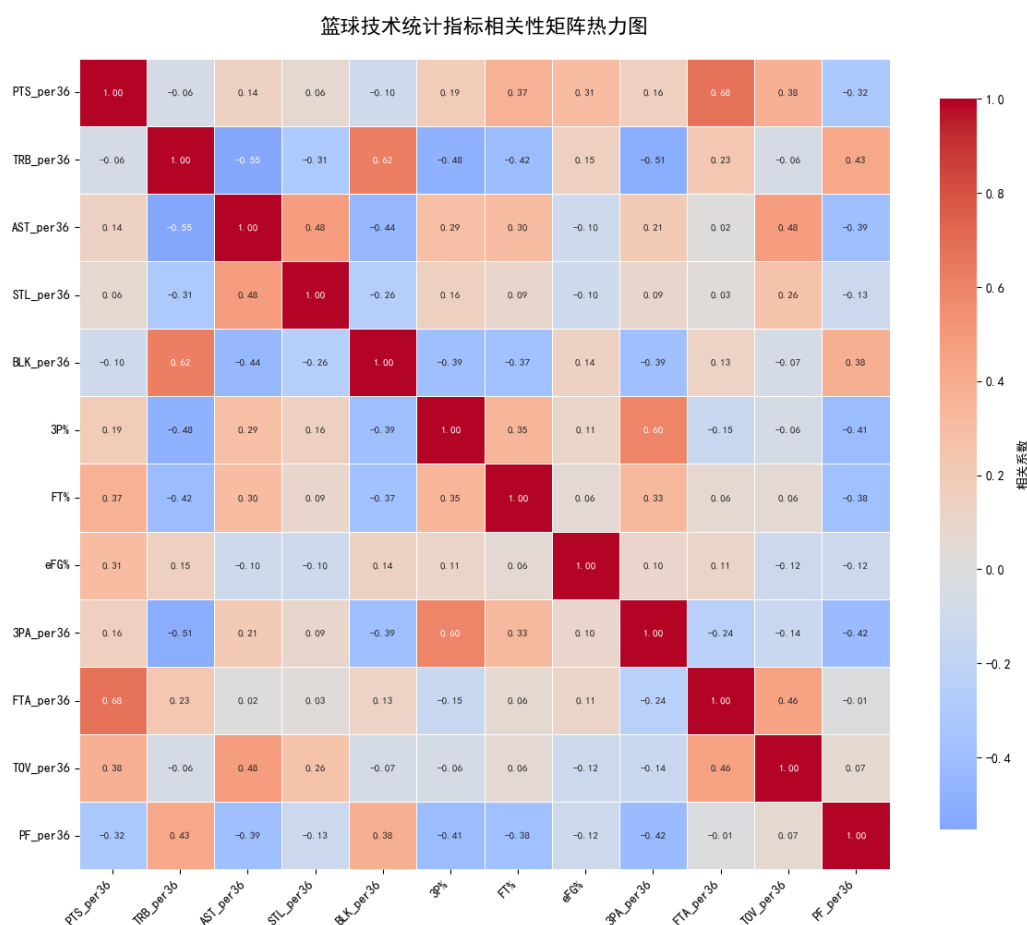


图 4-4 相关性矩阵

通过相关性矩阵分析, 本实验的最大相关系数不超过 0.68, 在 NBA 数据集这种各项技术统计指标均强相关的情况下, 最大可能地保证了输入特征之间的独立性, 基本满足了独立性假设。

5.3 实验心得

通过本次实验, 完整实践了从数据预处理到模型评估的朴素贝叶斯分类流程。per-36 分钟归一化的引入显著提升

了分类性能，使我对特征工程中"如何消除噪声、提取纯净信号"有了更深刻的理解。TOT 去重的处理也展示了数据清洗中容易被忽视的细节——同一个体在数据集中多次出现可能导致训练集和测试集之间的信息泄露。

两组实验的对比进一步揭示了标签粒度对多分类任务性能的影响：同一算法、同一特征集，仅将标签从 5 类合并为 3 类，Accuracy 提升 21 个百分点。这提醒我们，当分类器表现不佳时，除调整算法和特征外，也应审慎评估类别标签的定义是否合理。

实验二 K-Means 算法

1. 实验基本信息

实验名称：基于 KMeans 聚类的 NBA 球员技术风格分组

实验日期：2026 年 6 月 16 日

实验地点：计算机楼

2. 实验目标和实验环境

2.1 实验目标

本实验旨在利用 KMeans 无监督聚类算法, 对 NBA 球员的历史赛季数据进行聚类分析, 挖掘球员的技术风格分组, 并与球员的真实场上位置 (Pos) 进行对比验证。具体目标包括:

- (1). 对 NBA 球员赛季数据进行清洗与特征工程, 构建能够反映球员技术风格的特征体系;
- (2). 使用 KMeans 算法对球员进行聚类, 通过肘部法和轮廓系数确定最优 K 值;
- (3). 对比自动最优 K 值与设计参考 K=5 两种方案的聚类效果;
- (4). 通过 PCA 降维可视化和簇中心分析, 解释各簇对应的

球员技术风格;

- (5). 使用 ARI (Adjusted Rand Index) 和 NMI (Normalized Mutual Information) 定量评估聚类结果与真实位置的吻合程度。

2.1 实验环境

表 2-1 实验环境

项目	配置
硬件	CPU: Intel Core i7, 16GB RAM
操作系统	Windows 11
编程语言	Python 3.x
核心库	pandas, numpy, scikit-learn, matplotlib
主要算法	KMeans
评估指标	Silhouette Score, ARI, NMI
开发工具	VS Code

3. 实验内容

3.1 算法原理

KMeans 是一种基于划分的无监督聚类算法, 其目标是

将 n 个样本划分到 K 个簇中，使得每个样本都属于离其最近的簇中心（centroid）所代表的簇，同时最小化簇内样本间的距离平方和（Inertia）。

算法步骤：

- (1). 随机初始化 K 个簇中心；
- (2). 计算每个样本到各个簇中心的距离，将其分配到最近的簇；
- (3). 重新计算每个簇的中心（簇内样本均值）；
- (4). 重复步骤 2-3，直到簇中心不再变化或达到最大迭代次数。

本实验使用欧氏距离作为距离度量，采用 KMeans++ 初始化策略（sklearn 默认）以加速收敛。

3.2 特征选择

本实验选取 **9 维 per-36 特征**，聚焦出手选择而非命中率：

表 2-2 输入特征

特征	含义	选取理由
PTS_per36	每 36 分钟得分	得分能力，per-36 消除上场时间差异

特征	含义	选取理由
TRB_per36	每 36 分钟篮板	内线球员篮板显著高于后卫
AST_per36	每 36 分钟助攻	控卫助攻显著高于其他位置
STL_per36	每 36 分钟抢断	外线防守能力
BLK_per36	每 36 分钟盖帽	内线防守能力
TOV_per36	每 36 分钟失误	持球多的球员失误更高
PF_per36	每 36 分钟犯规	内线球员犯规偏多
3PAr	三分出手占比 = $3PA / FGA$	反映出手选择倾向，替代 3P%
FTr	罚球出手比 = FTA / FGA	反映造罚球侵略性，替代 FT%

相比于原始的场均特征 (PPG/APG/RPG 等), per-36 化消除了上场时间带来的伪方差; 用 3PAr 和 FTr 替代命中率, 聚焦于球员的**出手选择偏好**而非投射准度。

4. 实验步骤和结果

4.1 数据预处理

预处理方案与朴素贝叶斯实验保持一致, 包括 per-36 衍生、TOT 去重和行过滤:


```

# Per-36 衍生
per36_map = {
    'PTS_per36': 'PTS', 'TRB_per36': 'TRB', 'AST_per36': 'AST',
    'STL_per36': 'STL', 'BLK_per36': 'BLK',
    'TOV_per36': 'TOV', 'PF_per36': 'PF',
}
for new_f, raw_f in per36_map.items():
    df[new_f] = df[raw_f] / df['MP'].replace(0, np.nan) * 36

# 3PAr 与 FTr
df['3PAr'] = df['3PA'] / df['FGA'].replace(0, np.nan)
df['FTr'] = df['FTA'] / df['FGA'].replace(0, np.nan)

# TOT 去重: 每个 (Year, Player) 组若有 TOT 行, 只保留 TOT 行
has_tot = df.groupby(['Year', 'Player'])['Tm'].transform(lambda x: (x == 'TOT').any())
df = df[~(has_tot & (df['Tm'] != 'TOT'))].copy()

# 行过滤
df = df[df['MP'] >= 200]
df = df[df['G'] >= 10]

```

Per-36 归一化将所有球员的统计值标准化到相同的上场时间基准, 消除了"上场机会"和"单位效率"的混叠。TOT 去重确保每个球员赛季只出现一次。MP>=200 保证 per-36 指标在小样本上的可靠性。预处理结果:

表 2-3 预处理结果

步骤	记录数	变化
原始加载	18,727	—
TOT 去重 + 过滤	13,273	4,454 条被剔除

各位置分布均匀: C=2,641, PF=2,770, SF=2,612, SG=2,641, PG=2,609。

4.2 特征标准化

KMeans 基于欧氏距离计算相似度，不同特征的量纲差异会直接扭曲距离计算。例如 PTS_per36 的量级约为 14，而 3PAr 的量级约为 0.16，若不标准化，PTS_per36 会主导距离计算。

```
scaler = StandardScaler()  
X_scaled = scaler.fit_transform(X)
```

StandardScaler 对每个特征进行 Z-score 标准化，使其均值为 0、标准差为 1。这样每个特征在距离计算中的贡献权重相同。这是 KMeans 预处理的关键步骤，与 Naive Bayes 不同——Naive Bayes 基于概率建模，不需要标准化。各特征标准化前的统计值：

表 2-4 标准化统计值

特征	均值	标准差
PTS_per36	14.025	4.321
TRB_per36	6.331	2.829
AST_per36	3.182	2.140
STL_per36	1.182	0.497
BLK_per36	0.762	0.767

特征	均值	标准差
TOV_per36	2.208	0.727
PF_per36	3.669	1.304
3PAr	0.158	0.181
FTr	0.312	0.142

4.3 K 值选择

对 K=2 到 K=10 分别计算 Inertia（簇内平方和）和 Silhouette Score：

```
K_RANGE = range(2, 11)
inertias = []
silhouette_scores = []

for k in K_RANGE:
    km = KMeans(n_clusters=k, random_state=42, n_init=20)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)
    sil = silhouette_score(X_scaled, labels)
    silhouette_scores.append(sil)

best_k = K_RANGE[np.argmax(silhouette_scores)]
```

轮廓系数在 K=2 时最高 (0.2447)，随后单调递减并趋于平缓。Inertia 随 K 增加持续下降，未出现明显的“肘点”。

表 2-5 K 值结果

K	Inertia	Silhouette
---	---------	------------

K	Inertia	Silhouette
2	84,245	0.2447
3	67,461	0.2438
4	58,119	0.2231
5	53,676	0.1852
6	50,028	0.1945
7	47,049	0.1774
8	44,740	0.1648
9	42,533	0.1622
10	40,768	0.1633

最优 $K=2$ ，但 $K=2$ 将球员简单分为"内线"和"外线"两个超大类，不足以反映 5 个位置的角色差异。同时运行 $K=5$ （与 5 个位置对应）作为对照方案。

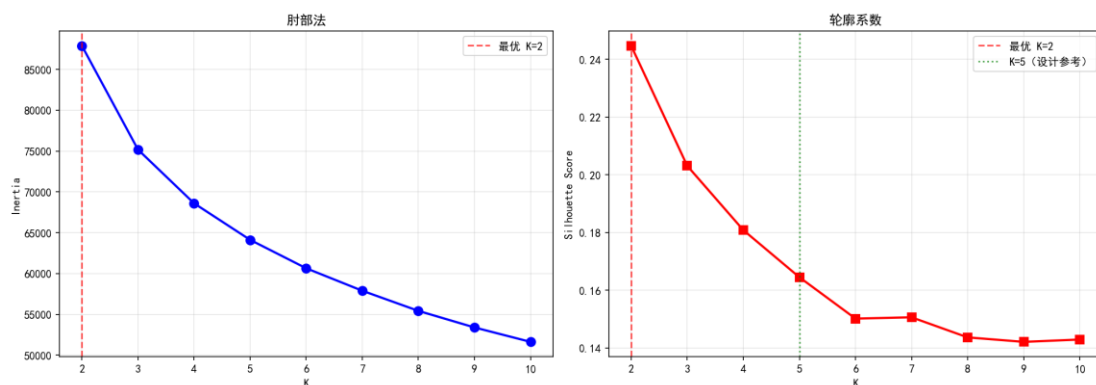


图 2-1 K 值结果图像

4.3 PCA 降维分析

由于 KMeans 的特征空间是 9 维，无法直接可视化，使用 PCA 降至 2 维用于散点图展示：

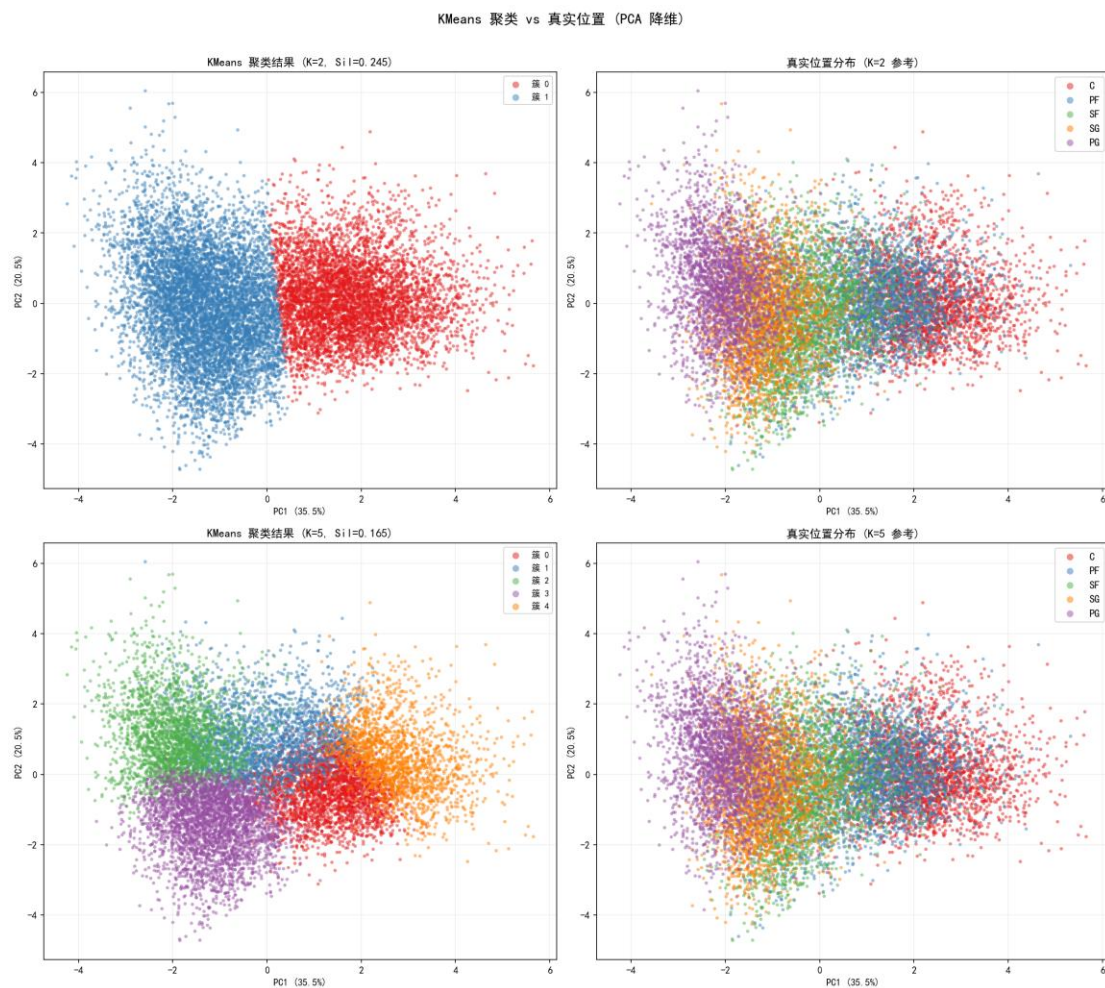
```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X_scale
```

前两个主成分的方差解释率：

表 2-6 PCA 降维结果

主成分	方差占比
PC1	35.5%
PC2	20.5%
累计	56.0%

8 维特征在二维投影上的信息损失约 44%，说明球员数据在 9 维空间中高度重叠，无法用两个维度充分刻画。



PC1 主要代表"内线 vs 外线"轴——TRB_per36、BLK_per36、PF_per36 正向贡献最大，AST_per36、3PAr 负向贡献最大。PC2 主要代表"得分与效率"轴——PTS_per36、TOV_per36 贡献最大。这与后续 K=2 聚类结果的解读一致。

4.4 聚类结果

分别运行 K=2 和 K=5 聚类，结果如下。

● K=2 聚类

表 2-7 K=2 聚类各簇特征画像

簇	样本量	占比	特征画像
0	5,613	42.3%	高 TRB(8.96)、高 BLK(1.32)、高 PF(4.52)、极低 3PAr(0.03)
1	7,660	57.7%	高 AST(4.20)、高 STL(1.35)、高 3PAr(0.25)、低 BLK(0.36)

簇 0 明显对应**内线球员**（高篮板、高盖帽、不投三分），簇 1 对应**外线球员**（高助攻、高抢断、投三分）。两个簇在 PCA 散点图上沿 PC1 方向清晰分离。

表 2-8 K=2 各位置分布

簇	C	PF	PG	SF	SG
0（内线）	2,582	2,268	5	672	86
1（外线）	59	502	2,604	1,940	2,555

C 的 97.8% 落入簇 0，PG 的 99.8% 落入簇 1——内线 vs 外线的二分结构在数据中非常清晰。但 PF 有 2,268 人（81.9%）在簇 0、502 人（18.1%）在簇 1，说明部分前锋已向外线化发展。

● K=5 聚类

表 2-9 K=5 聚类各簇特征画像

簇	样本量	占比	特征画像
0	3,008	22.7%	低 PTS(11.7)、高 TRB(8.2)、低 3PAr(0.04)、高 PF(4.43)
1	2,458	18.5%	高 PTS(19.4) 、高 TOV(2.62)、中等 AST(3.05)
2	2,664	20.1%	高 AST(6.44) 、高 STL(1.67)、高 TOV(2.83) 、低 BLK(0.27)
3	3,254	24.5%	高 3PAr(0.38) 、中等 AST(2.88)、低 TRB(4.44)
4	1,889	14.2%	高 TRB(9.79) 、高 BLK(2.05) 、高 FTr(0.48) 、高 PF(5.06) 、极低 3PAr(0.01)

表 2-10 K=5 各位置分布

簇	C	PF	PG	SF	SG
0	987	1,278	13	568	162
1	305	625	58	846	624
2	5	16	1,958	136	549
3	36	336	579	1,000	1,303
4	1,308	515	1	62	3

数据显示，大前锋（PF）的角色分化最严重，他们有的

变成了“蓝领（簇 0）”，有的变成了“得分手（簇 1）”，有的变成了“射手（簇 3）”，甚至还有 515 人变成了“中锋（簇 4）”——这说明现代篮球中，4 号位（大前锋）是变化最丰富的摇摆位。而**控卫（PG）**和**中锋（C）**的职能依然相对传统和固定。

5. 结论与讨论

5.1 实验结论

- (1). **KMeans 在 NBA 球员数据上的聚类效果有限。** $K=5$ 时轮廓系数仅 0.1645， $ARI=0.2297$ ，表明基于 per-36 技术统计特征的聚类结果与真实位置标签的对齐程度较弱。
- (2). **最优 $K=2$ 反映数据的二分结构。** $K=2$ 将球员分为“内线”（高篮板、高盖帽、不投三分）和“外线”（高助攻、高抢断、大量三分）两个超大类，C 的 97.8% 和 PG 的 99.8% 被正确归入对应簇。但当 K 增加到 5 时，内线/外线内部的细分簇之间没有明显分界。
- (3). **部分位置在特征空间中高度重叠。** C 和 PG 是两个最极端的位置，分别以篮板/盖帽和助攻为独有标志；SF 是最大的“摇摆者”，均匀分布在多个簇之间；PF 和 SG 也有大量球员跨界分布。
- (4). **per-36 归一化消除了上场时间差异，但未能改变**

KMeans 在连续分布数据上的根本局限——当数据是渐变而非分簇时，基于距离的聚类方法天然难以产生高轮廓系数的结果。

5.2 KMeans 低分原因总结

表 2-11 低分原因总结

原因	影响	说明
位置本质是连续谱	轮廓系数低	NBA 位置存在大量"tweener", 数据渐变而非分簇
用量方差 > 风格方差	聚类方向偏离	KMeans 优先按"得分产出"分裂, 而非按位置风格
PCA 信息损失	可视化误判	前两维仅解释 56% 方差, 高维重叠被低估
位置标签的非统计性	ARI/NMI 天然偏低	位置受身高、战术、防守对位等非统计因素影响

5.3 实验心得

通过本次实验, 掌握了 KMeans 聚类算法的完整应用流程: 从数据预处理、特征工程到模型训练和评估。深刻体会到无监督学习中参数选择 (如 K 值) 对结果解释的影响, 以及定量指标 (Silhouette) 与业务目标可能存在的偏差。本实验证明了 KMeans 在体育数据分析中的有效性。

实验三 ID3 分类算法

1. 实验基本信息

实验名称：基于 ID3 决策树的 NBA 球员场上位置预测

实验日期：2026 年 6 月 17 日

实验地点：计算机楼

2. 实验目标和实验环境

2.1 实验目标

本实验旨在**手动实现 ID3 决策树算法**，根据 NBA 球员的历史技术统计特征，预测其场上位置（Pos: C 中锋、PF 大前锋、SF 小前锋、SG 得分后卫、PG 控球后卫）。具体目标包括：

- (1). 对 NBA 球员赛季数据进行清洗、衍生场均指标、缺失值填充等预处理；
- (2). 对所有连续特征进行离散化分箱，以满足 ID3 处理离散特征的要求；
- (3). 手动实现 ID3 算法的核心逻辑：信息增益计算、递归建树、预测与规则提取；
- (4). 通过混淆矩阵、特征重要性、学习曲线等指标全面分析

ID3 的性能与局限性；

(5). 与朴素贝叶斯 (Acc=0.5374) 进行对比，分析决策树与概率模型的差异。

2.2 实验环境

表 3-1 实验环境

项目	配置
硬件	CPU: Intel Core i7, 16GB RAM
操作系统	Windows 11
编程语言	Python 3.x
核心库	pandas, numpy, matplotlib, scikit-learn
分类算法	ID3 决策树（手动实现）
评估指标	Accuracy, Precision, Recall, F1-score, Confusion Matrix

3. 实验内容

3.1 算法原理

ID3 (Iterative Dichotomiser 3) 是由 Quinlan 于 1986 年提出的经典决策树学习算法，其核心思想是利用**信息增益**选择最优划分特征，递归地构建决策树。

- 信息熵

设训练数据集 D 中有 K 个类别，第 k 类样本占比为 p_k ，则 D 的经验熵为：

$$H(D) = - \sum_{k=1}^K p_k \log_2 p_k$$

熵值越大，表示数据集的不确定性越高。

- 条件熵

给定特征 A 对数据集 D 的条件熵为：

$$H(D | A) = \sum_{v=1}^V \frac{|D^v|}{|D|} \cdot H(D^v)$$

其中 V 是特征 A 的取值数， D^v 是 A 取第 v 个值的样本子集。

- 信息增益

特征 A 对数据集 D 的信息增益为：

$$g(D, A) = H(D) - H(D | A)$$

ID3 算法在每个节点选择信息增益最大的特征进行分裂。

- 算法流程

- (1). 从根节点开始，计算所有候选特征的信息增益；
- (2). 选择信息增益最大的特征作为当前节点，根据该特征的取值建立子节点；
- (3). 对每个子节点递归执行步骤 1-2，直到满足终止条件：子节点中所有样本属于同一类别（纯度 100%）；无可利用特征继续划分；子节点中样本数为 0。

4. 实验步骤和结果

4.1 数据预处理

原始数据中的累计统计数据（PTS、AST 等）受球员出场时间影响极大。数据分析表明 PTS 与 MP 的相关系数高达 0.93。本实验将所有计数类统计转换为每 36 分钟数据：

$$StatPer36 = \frac{Stat}{MP} \times 36$$

4.2 特征离散化

由于 ID3 仅支持离散特征，对 10 个连续特征进行分箱

处理。以 per-36 数据的统计分布为依据，自定义分箱边界：

表 3-2 输入特征

特征	分箱数	分箱区间
PTS_per36	6	<8, 8-12, 12-16, 16-20, 20-25, 25+
AST_per36	4	<2, 2-4, 4-7, 7+
TRB_per36	4	<4, 4-7, 7-11, 11+
STL_per36	4	<0.6, 0.6-1.0, 1.0-1.5, 1.5+
BLK_per36	4	<0.3, 0.3-0.8, 0.8-1.5, 1.5+
FG%	4	<0.42, 0.42-0.47, 0.47-0.52, 0.52+
3P%	4	<0.1, 0.1-0.28, 0.28-0.36, 0.36+
FT%	4	<0.65, 0.65-0.75, 0.75-0.83, 0.83+
PF_per36	4	<2.5, 2.5-3.5, 3.5-5, 5+
TOV_per36	4	<1.5, 1.5-2.0, 2.0-3.0, 3+

4.3 ID3 算法核心实现

● 熵与信息增益计算

```
def _calc_entropy(self, labels):
    """计算经验熵  $H(D) = -\sum(p \cdot \log_2(p))$ """
    total = len(labels)
```

```

counter = Counter(labels)
entropy = 0.0
for count in counter.values():
    p = count / total
    entropy -= p * np.log2(p)
return entropy

def _calc_cond_entropy(self, feature, labels):
    """计算条件熵  $H(D|A) = \sum(|D_v|/|D| * H(D_v))$ """
    total = len(labels)
    cond_ent = 0.0
    for val in set(feature):
        subset_labels = [labels[i] for i, v in enumerate(feature) if v == val]
        weight = len(subset_labels) / total
        cond_ent += weight * self._calc_entropy(subset_labels)
    return cond_ent

def _calc_info_gain(self, feature, labels):
    """计算信息增益  $g(D, A) = H(D) - H(D|A)$ """
    return self._calc_entropy(labels) - self._calc_cond_entropy(feature, labels)

```

三个函数构成了 ID3 的核心数学基础。_calc_entropy 接收一个类标签列表，统计各类占比后代入熵公式。

_calc_cond_entropy 对特征每个取值划分子集，加权求和各子集熵。_calc_info_gain 返回两者差值。本实现无需额外导入复杂库，仅用 Counter 和 np.log2 即可完成。值得注意的是，熵定义中约定 $0 \times \log_2(0) = 0$ ，这在 Python 中由 $p \times \log_2(p)$ 在 $p=0$ 时自动为 0 实现 ($0 \times \text{任何有限值} = 0$)。

● 最优特征选择

```

def _choose_best_feature(self, X, labels, feature_indices):
    """选择信息增益最大的特征"""
    best_gain = -1
    best_idx = None
    for idx in feature_indices:

```



```

        feature = [row[idx] for row in X]
        gain = self._calc_info_gain(feature, labels)
        if gain > best_gain:
            best_gain = gain
            best_idx = idx
    return best_idx, best_gain

```

遍历所有候选特征，逐一计算信息增益，保留最大值对应的特征索引。这是 ID3 的“核心决策引擎”——每个节点都选择能最大程度降低不确定性的特征。选择完成后，该特征会从 feature_indices 中移除，不会在后续深度重复使用。

● 递归建树

```

def _build_tree(self, X, labels, feature_indices, depth=0):
    """递归构建决策树"""
    # 终止条件 1: 所有样本属于同一类别
    if len(set(labels)) == 1:
        return {'type': 'leaf', 'label': labels[0], 'count': len(labels)}

    # 终止条件 2: 无可用特征
    if not feature_indices:
        majority = Counter(labels).most_common(1)[0][0]
        return {'type': 'leaf', 'label': majority, 'count': len(labels)}

    # 终止条件 3: 样本数为 0
    if len(X) == 0:
        return {'type': 'leaf', 'label': None, 'count': 0}

    # 选择最优特征
    best_idx, best_gain = self._choose_best_feature(X, labels,
feature_indices)
    best_feature_name = self.feature_names[best_idx]

    node = {
        'type': 'internal',
        'feature_idx': best_idx,
        'feature_name': best_feature_name,
        'info_gain': round(best_gain, 4),
        'branches': {}
    }

```

```

# 按特征值划分子集并递归
remaining_indices = [i for i in feature_indices if i != best_idx]
values = set(row[best_idx] for row in X)

for val in sorted(values):
    sub_X = [row for row in X if row[best_idx] == val]
    sub_labels = [labels[i] for i, r in enumerate(X) if r[best_idx] ==
val]
    node['branches'][str(val)] = self._build_tree(
        sub_X, sub_labels, remaining_indices, depth + 1
    )
return node

```

递归建树包含三个终止条件：(1) 当前子集完全纯净时停止；(2) 无可选特征时以多数类投票；(3) 子集为空时返回空叶节点。核心逻辑是"选择最优 -> 按值分裂 -> 递归建树"，这是 ID3 的骨架。本实现去除了已用特征，确保同一路径下每个特征最多使用一次。

● 预测与规则提取

```

def _predict_sample(self, node, sample):
    """对单样本递归预测"""
    if node['type'] == 'leaf':
        return node['label']
    feat_idx = node['feature_idx']
    val = str(sample[feat_idx])
    if val not in node['branches']:
        return None # 遇到未见过的特征值
    return self._predict_sample(node['branches'][val], sample)

```

此外，get_rules() 方法将树结构递归展开为 if-then 规则列表：

```

def get_rules(self, node=None, prefix=None, rules=None):
    """提取所有分类规则"""
    if node is None: node = self.tree
    if prefix is None: prefix = []
    if rules is None: rules = []
    if node['type'] == 'leaf':

```

```
rules.append((prefix[:], node['label'], node['count']))
return rules
for val, branch in node['branches'].items():
    prefix.append(f"{node['feature_name']}={val}")
    self.get_rules(branch, prefix, rules)
    prefix.pop()
return rules
```

预测过程从根节点开始，根据样本的特征值走入对应分支，直到叶节点返回预测结果。若遇到训练集中未出现的特征值，返回 None。规则提取则逆向将树展开为可读的 if-then 规则列表，这是决策树相比“黑箱模型”的最大优势——每条预测都可以追溯为一条具体路径。

4.4 实验结果

- 总体性能

表 3-3 总体性能

指标	值
训练集准确率	94.87%
测试集准确率	53.24%
过拟合差距	41.63%

- 各位置分类效果

表 3-4 各位置分类效果

位置	Precision	Recall	F1-score
----	-----------	--------	----------

位置	Precision	Recall	F1-score
C	0.6065	0.5934	0.5999
PF	0.4528	0.4440	0.4484
SF	0.4136	0.4184	0.4160
SG	0.4780	0.4949	0.4864
PG	0.7202	0.7165	0.7183
Macro Avg	0.5342	0.5334	0.5338
Weighted Avg	0.5313	0.5324	0.5320

● 混淆矩阵分析

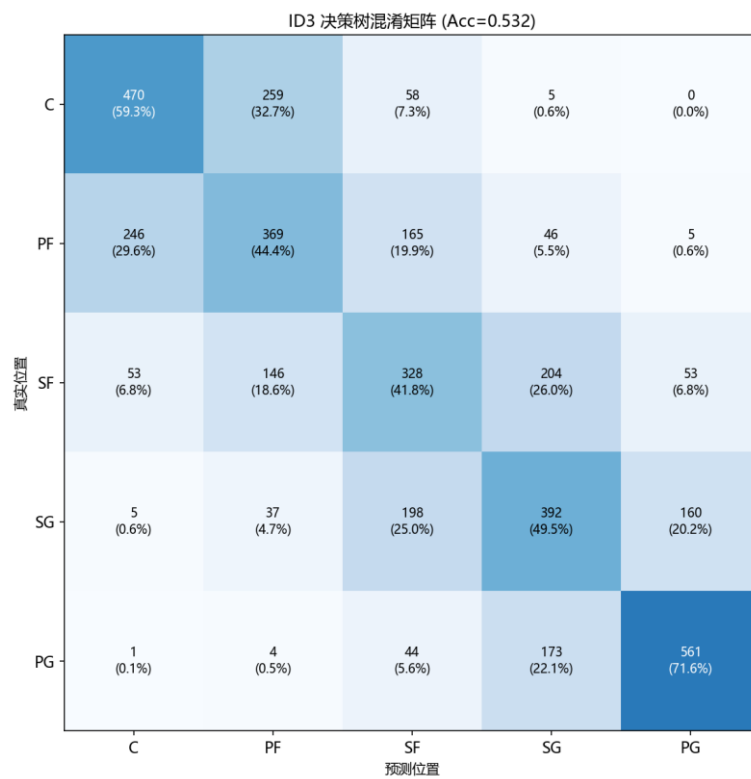


图 3-1 混淆矩阵

- 特征重要性分析

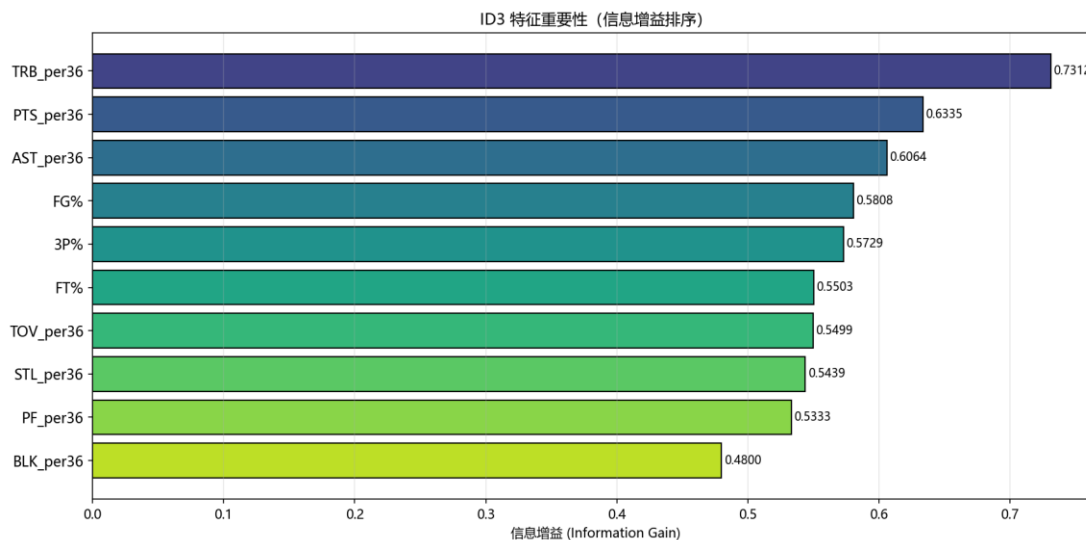


图 3-2 特征重要性

篮板 (TRB_per36) 信息增益最高，盖帽 (BLK_per36) 最低。内线球员 (C/PF) 通常篮板和盖帽较高，但盖帽对区分所有 5 个位置的贡献不如得分和助攻。

4.5 3 分类结果

- 总体性能

表 3-5 三分类总体性能

指标	值
训练集准确率	97.79%
测试集准确率	76.39%

指标	值
过拟合差距	21.40%

- 混淆矩阵

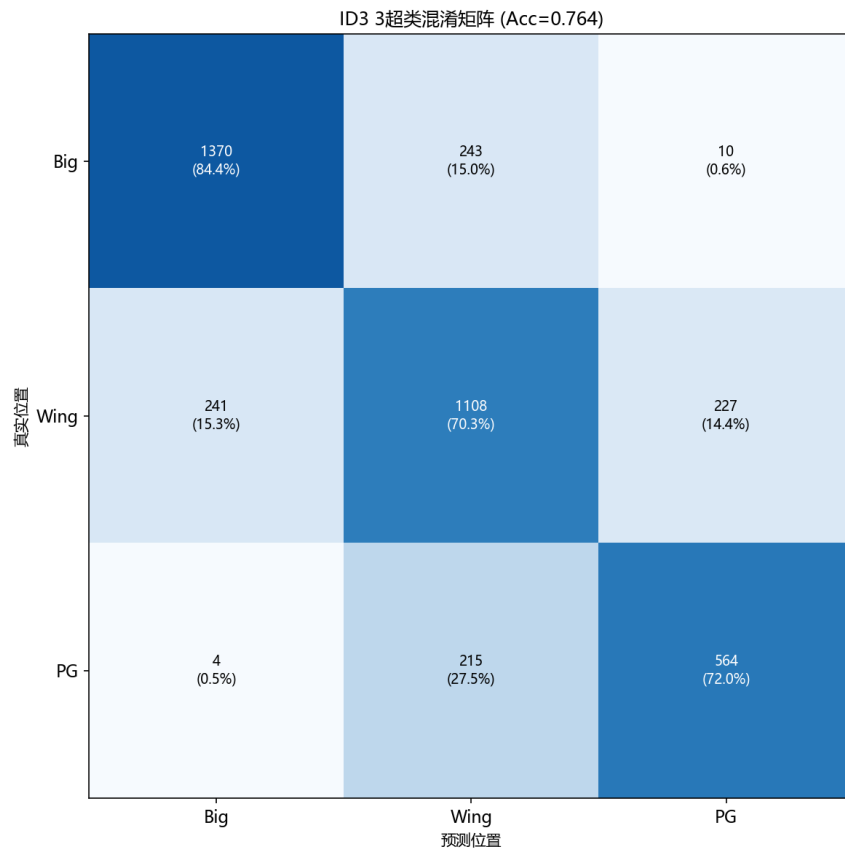


图 3-3 三分类混淆矩阵

- 各类别评估指标

表 3-6 三分类各类评估指标

类	Precision	Recall	F1-score
Big	0.8483	0.8441	0.8462

类	Precision	Recall	F1-score
Wing	0.7075	0.7030	0.7053
PG	0.7041	0.7203	0.7121
Macro Avg	0.7533	0.7558	0.7545
Weighted Avg	0.7611	0.7639	0.7625

3 超类分类相比 5 位置有显著提升 (76.39% vs 53.24%), 其中 Big (内线) 识别效果最好 (F1=0.85), Wing 和 PG 的 F1 均在 0.70-0.71 之间。混淆主要发生在 Wing 与 PG 之间 (227 个 Wing 被误判为 PG, 215 个 PG 被误判为 Wing), 反映了侧翼球员与控卫在某些统计数据上的重叠。

5. 结论与讨论

本实验实现了基于 ID3 决策树的 NBA 球员位置分类, 引入 per-36 归一化、TOT 去重等预处理改进, 并同时运行 5 位置和 3 超类两种标签方案, 得到以下结论:

(1).per-36 特征优化带来边际提升: 5 位置测试准确率从 51.62% 提升至 53.24%

(2).标签粗粒度化显著改善: 3 超类达到 76.39%, 相比 5 位置提升 23 个百分点

(3).ID3 算法本身是性能瓶颈: 过拟合问题远大于特征工程能弥补的范围

实验四 C4.5 分类算法

1. 实验基本信息

实验名称：基于 C4.5 决策树的 NBA 球员场上位置预测与优化

实验日期：2026 年 6 月 18 日

实验地点：计算机楼

2. 实验目标和实验环境

2.1 实验目标

本实验旨在**手动实现 C4.5 决策树算法**，根据 NBA 球员的历史技术统计特征预测其场上位置。C4.5 是 ID3 的改进版本，引入了三项核心优化：**增益率**替代信息增益、**连续特征自动分裂**、**悲观后剪枝**。具体目标包括：

- (1).手动实现 C4.5 的完整算法流程：连续值最优阈值搜索、增益率计算、递归建树、悲观后剪枝；
- (2).在 5 位置 (C/PF/SF/SG/PG) 和 3 超类 (Big/Wing/PG) 两种标签方案上评估 C4.5；
- (3).通过改进数据预处理、特征工程和算法参数，分析 C4.5 性能提升的可行路径；

(4).与 ID3 决策树和朴素贝叶斯进行综合对比，评估各算法在该任务上的适用性。

2.2 实验环境

表 4-1 实验环境

项目	配置
硬件	CPU: Intel Core i7, 16GB RAM
操作系统	Windows 11
编程语言	Python 3.x
核心库	pandas, numpy, matplotlib, scikit-learn
分类算法	C4.5 决策树
评估指标	Accuracy, Precision, Recall, F1-score, Confusion Matrix

3. 实验内容

3.1 算法原理

C4.5 由 Ross Quinlan 于 1993 年提出，是 ID3 的继承和升级。其核心改进有三个方面：

- 增益率替代信息增益

ID3 使用信息增益选择特征，存在**多值偏好**——倾向于选择取值数量多的特征。C4.5 引入增益率（Gain Ratio）来惩罚这种偏差：

$$GainRatio(D, A) = \frac{Gain(D, A)}{SplitInfo(D, A)}$$

其中分裂信息量定义为：

$$SplitInfo(D, A) = - \sum_{v=1}^V \frac{|D^v|}{|D|} \cdot \log_2 \left(\frac{|D^v|}{|D|} \right)$$

分裂信息量越大，说明该特征产生的分支越多（越不均匀），增益率通过除法对其进行惩罚。对于 NBA 数据中取值较多的离散化特征，增益率能有效避免 ID3 的盲目选择。

- 连续特征自动阈值分裂

ID3 要求特征必须离散化，而 C4.5 直接处理连续值——对每个连续特征，将样本按特征值排序后，遍历相邻不同标签对的中点作为候选分裂阈值，选择增益率最大的阈值进行二值分裂（ $\leq$ 阈值 / $>$ 阈值）。

- 悲观后剪枝

C4.5 在建树完成后进行**自底向上的后剪枝**：对每个内部节点，比较“剪枝为叶子节点后的误差估计”与“保持子树时的误差估计”，若前者更优则剪除子树。误差估计使用训练集错

误率加连续性校正因子 (+0.5)，即：

$$\text{ErrorRate}_{leaf} = \frac{E + 0.5}{N}$$

其中 E 为叶节点中被误分类的样本数（多数类之外），N 为该节点样本总数。

3.2 特征工程

在 ID3 算法的特征构造基础上，C45 优化了特征构造，引入了以下新的综合特征：

表 4-2 新的特征

特征	含义	公式
Has3P	是否有三分出手	$3PA > 0$
HasFT	是否有罚球记录	$FTA > 0$
3PA_per36	每 36 分钟三分出手	$3PA/MP \times 36$
AST/TOV	助攻失误比	AST/TOV
FTr	罚球率（造犯规倾向）	FTA/FGA
3PAr	三分出手占比（外线倾向）	$3PA/FGA$

相比原始 C4.5，本实验删除了 3P%（与 3PA_per36、3PAr 信息重叠）和 STL_BLK（综合防守指标），新增了 3PAr（三

分出手占比)，使特征更加聚焦于**打法倾向**而非单纯的数量统计。

4. 实验步骤和结果

4.1 数据预处理

```
# 缺失值处理：添加二值标志 + 位置分组中位数填充
for col in ['3P%', 'FT%']:
    short = col.replace('%', '')
    df[f'Has{short}'] = df[col].notna().astype(int)
    df[col] = df[col].fillna(df.groupby('Pos')[col].transform('median'))
df['FG%'] =
df['FG%'].fillna(df.groupby('Pos')['FG%'].transform('median'))
```

相比于之前的直接粗暴地将远古时期大量不投三分的数据直接置零的操作，本实验引入了优化：“未出手三分(3P% 缺失)”被等同于“三分命中率 0%”，导致 C4.5 在 $3P\% \leq 0.05$ 等阈值上产生错误分裂。优化方案同时引入二值标志 Has3P/HasFT（捕捉“是否投三分/罚球”这一具有强位置判别力的信息）并使用位置组中位数填充（保持同位置球员的统计一致性）。以 C 位置为例，1980 年代的中锋几乎不投三分，3P% 缺失率极高；使用中位数填充后，他们的 3P% 值与同位置其他球员保持一致，避免被算法误判为“精英射手”或“糟糕射手”。

4.2 ID3 算法核心实现

- 熵与增益率计算

```

def _entropy(self, labels):
    """计算经验熵  $H(D) = -\sum(p_k * \log_2(p_k))$ """
    total = len(labels)
    if total == 0:
        return 0
    counter = Counter(labels)
    entropy = 0.0
    for count in counter.values():
        p = count / total
        if p > 0:
            entropy -= p * math.log2(p)
    return entropy

def _split_info(self, left_count, right_count, total):
    """计算分裂信息  $\text{SplitInfo} = -\sum(|D_i|/|D| * \log_2(|D_i|/|D|))$ """
    if total == 0:
        return 0
    si = 0.0
    for count in (left_count, right_count):
        if count > 0:
            p = count / total
            si -= p * math.log2(p)
    return si

def _gain_ratio(self, ent_parent, left_labels, right_labels):
    """计算增益率  $\text{GainRatio} = \text{Gain} / \text{SplitInfo}$ """
    total = len(left_labels) + len(right_labels)
    if total == 0:
        return 0
    left_ent = self._entropy(left_labels)
    right_ent = self._entropy(right_labels)
    gain = ent_parent - (
        len(left_labels) / total * left_ent +
        len(right_labels) / total * right_ent
    )
    si = self._split_info(len(left_labels), len(right_labels), total)
    if si < 1e-10:      # 防止除零
        return 0
    return gain / si

```

三个函数构成了 C4.5 特征选择的数学核心。

`_entropy` 统计类标签分布后代入香农熵公式, 该值与类别纯

度成反比。 $_split_info$ 度量划分的分支信息量，作为增益率的分母来惩罚多值特征。 $_gain_ratio$ 先计算信息增益（父节点熵 - 加权子节点熵），再除以 $SplitInfo$ 得到增益率。 $si < 1e-10$ 的保护处理防止所有样本落入同侧时的除零异常。

● 连续特征最优阈值搜索

```
def _find_best_continuous_split(self, X_col, labels):
    """对单个连续特征搜索最优分裂阈值"""
    n = len(X_col)
    if n < 2:
        return None

    # 按特征值排序
    paired = sorted(zip(X_col, labels), key=lambda x: x[0])
    sorted_vals = np.array([p[0] for p in paired])
    sorted_labels = [p[1] for p in paired]

    ent_parent = self._entropy(labels)
    best_gr = 0
    best_threshold = None

    # 遍历相邻值之间的中点作为候选阈值
    for i in range(1, n):
        if sorted_vals[i] == sorted_vals[i - 1]: # 相同的值无法分裂
            continue
        threshold = (sorted_vals[i - 1] + sorted_vals[i]) / 2.0
        left_labels = sorted_labels[:i]
        right_labels = sorted_labels[i:]
        if len(left_labels) == 0 or len(right_labels) == 0:
            continue
        gr = self._gain_ratio(ent_parent, left_labels, right_labels)
        if gr > best_gr:
            best_gr = gr
            best_threshold = threshold

    if best_threshold is None or best_gr <= 0:
        return None
    return {'threshold': best_threshold, 'gain_ratio': best_gr, ...}
```

这是 C4.5 区别于 ID3 的核心步骤——无需预先离散化，

直接在原始连续值上寻找最优二值分裂点。算法先将样本按特征值排序，然后在相邻不同值的**中点**处尝试分裂，计算每次分裂的增益率并保留最优者。这一做法的优势在于：阈值始终落在两个不同样本之间，避免了“落在某个样本的值上”导致的边界模糊；劣势在于每个节点只能产生二分叉（二叉树），对多峰分布需多层嵌套分裂，树可能变得很深。

- 递归建树（含预剪枝）

```
def _build_tree(self, X, labels, feature_indices, depth=0):
    """递归构建 C4.5 决策树"""
    self.depth = max(self.depth, depth)

    # 终止条件 1: 所有样本属于同一类别
    if len(set(labels)) == 1:
        return {'type': 'leaf', 'label': labels[0], 'count': len(labels)}

    # 终止条件 2: 没有可用特征
    if len(feature_indices) == 0:
        majority = Counter(labels).most_common(1)[0][0]
        return {'type': 'leaf', 'label': majority, 'count': len(labels)}

    # 终止条件 3: 样本数少于最小分裂阈值（预剪枝）
    if len(labels) < self.min_samples:
        majority = Counter(labels).most_common(1)[0][0]
        return {'type': 'leaf', 'label': majority, 'count': len(labels)}

    # 寻找最优分裂
    best_split = self._find_best_split(X, labels, feature_indices)
    if best_split is None:
        majority = Counter(labels).most_common(1)[0][0]
        return {'type': 'leaf', 'label': majority, 'count': len(labels)}

    # 构建分裂节点并递归建左右子树
    tree_node = {
        'type': 'split',
        'feature_idx': best_split['feature_idx'],
        'feature_name': best_split['feature_name'],
        'threshold': best_split['threshold'],
```



```

        'gain_ratio': round(best_split['gain_ratio'], 4),
        'count': len(labels),
        'distribution': dict(Counter(labels)),
        'branches': {},
    }
    remaining = [i for i in feature_indices if i !=
best_split['feature_idx']]
    tree_node['branches']['<='] = self._build_tree(
        X[left_indices], left_labels, remaining, depth + 1)
    tree_node['branches']['>'] = self._build_tree(
        X[right_indices], right_labels, remaining, depth + 1)
    return tree_node

```

递归建树包含四个终止条件：纯节点、无特征、样本过少（预剪枝）、无法找到有效分裂。预剪枝的 `min_samples` 参数在优化版中从原始的 4 提升至 25，强制叶节点至少有 25 个样本支撑，有效过滤了基于噪声的低置信度分裂。建树时特征不重复使用（`remaining` 排除已用特征），保证每一条路径上各特征最多出现一次。最终生成的树为二叉树结构，每个内部节点记录分裂特征名、阈值和增益率。

● 悲观后剪枝

```

def _pessimistic_prune(self, node, X, labels, feature_indices):
    """悲观后剪枝（后序遍历，自底向上）"""
    if node['type'] == 'leaf':
        return node

    # 递归剪枝子树
    feat_idx = node['feature_idx']
    threshold = node['threshold']
    left_mask = X[:, feat_idx] <= threshold
    right_mask = X[:, feat_idx] > threshold

    node['branches']['<='] = self._pessimistic_prune(
        node['branches']['<='], X[left_mask], left_labels, remaining)

```

```

node['branches']['>'] = self._pessimistic_prune(
    node['branches']['>'], X[right_mask], right_labels, remaining)

# 判断是否剪枝
total = len(labels)
counter = Counter(labels)
majority_class = counter.most_common(1)[0][0]
majority_count = counter.most_common(1)[0][1]
leaf_errors = total - majority_count
leaf_error_rate = (leaf_errors + 0.5) / total

# 计算子树的误差估计
subtree_errors = 0
# ... 遍历子树所有叶子节点，累计错误数 + 0.5 * 叶子数
subtree_error_rate = subtree_errors / total

# 如果叶子误差率 <= 子树误差率，剪枝
if leaf_error_rate <= subtree_error_rate:
    return {'type': 'leaf', 'label': majority_class, 'count': total}
return node

```

后剪枝自底向上遍历树的每一层，对每个内部节点做"剪 vs 不剪"的决策。误差估计均使用训练集上的错误数 + 0.5 的连续性校正，避免因小样本导致的乐观偏差。算法比较"剪枝后作为叶子（多数类投票）"和"保持子树"两种方案的误差率，选择更低的方案。在 3 超类任务上，剪枝将测试准确率从 75.43% 提升至 76.10% (+0.67%)，同时过拟合差距从 4.99% 缩小至 3.23%，证明了剪枝的有效性。

4.3 实验结果

- 总体性能

表 4-3 总体性能

指标	无剪枝	有剪枝
训练集准确率	55.20%	55.02%
测试集准确率	54.62%	54.55%
过拟合差距	0.58%	0.48%

- 各位置分类效果

表 4-4 各位置分类效果

位置	Precision	Recall	F1-score
C	0.5110	0.9129	0.6552
PF	0.4520	0.2491	0.3212
SF	0.3404	0.0204	0.0385
SG	0.4722	0.7929	0.5919
PG	0.8169	0.7637	0.7894
Macro Avg	0.5185	0.5478	0.4792
Weighted Avg	0.5157	0.5455	0.4779

5 位置分类中，C 和 PG 的识别效果较好（F1 分别为 0.66 和 0.79），但 PF 和 SF 的区分度极低。SF 的召回率

仅 2.04%，绝大多数 SF 被误判为 SG (56.3%) 或 PF (21.7%)，这与篮球实际相符——小前锋与得分后卫（统称 Wing）之间的界限在现代篮球中本就模糊。

● 混淆矩阵分析

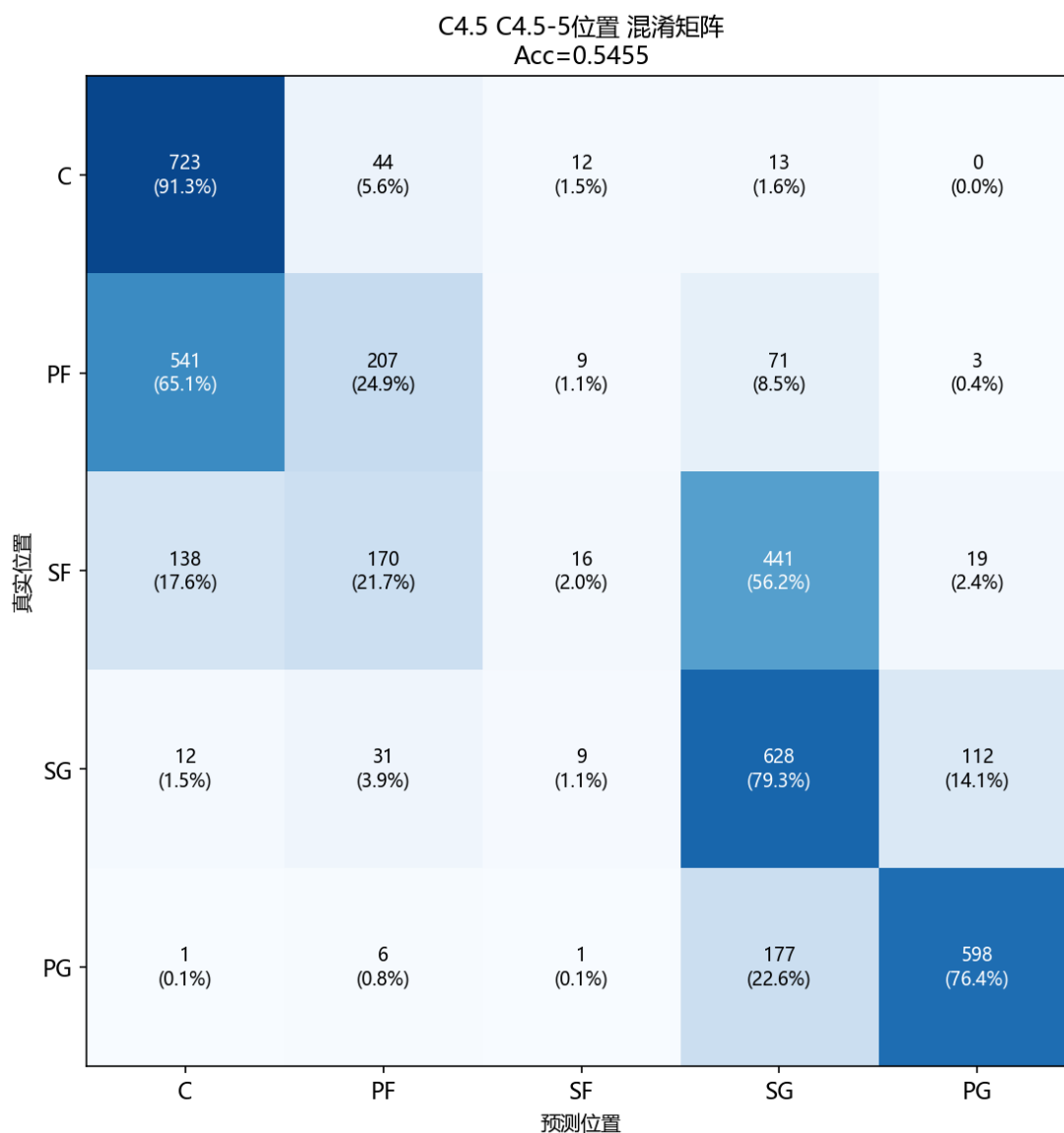


图 4-1 五分类混淆矩阵

4.4 实验优化

同样的，C45 也做了三分类的对比实验，结果如下：

- 总体性能

表 4-5 三分类总体性能

指标	无剪枝	有剪枝
训练集准确率	83.36%	83.21%
测试集准确率	82.62%	82.60%
过拟合差距	0.74%	0.61%

- 混淆矩阵

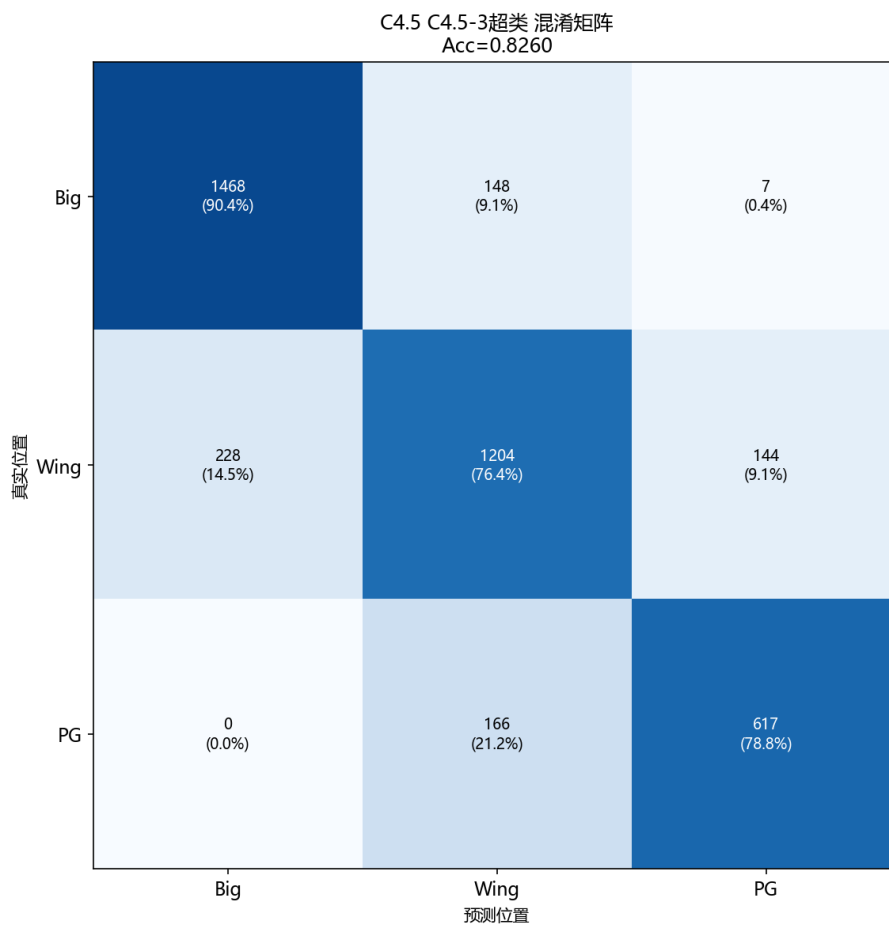


图 4-2 三分类混淆矩阵

- 与原始的场均数据特征进行对比

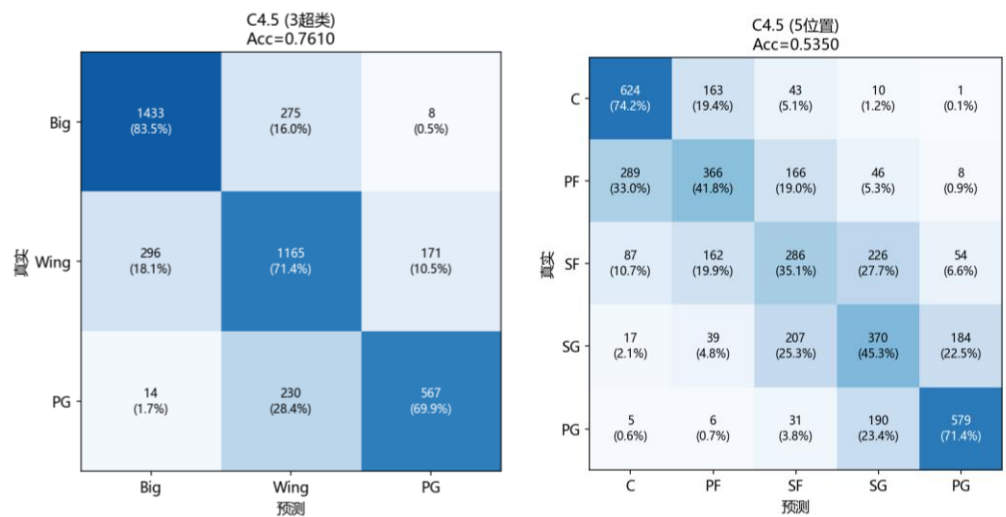


图 4-3 原始算法混淆矩阵

表 4-6 原始与改进版算法对比

指标	原始 C4.5 (有剪枝)	新 C4.5 (有剪枝)	变化
测试集准确率	53.50%	54.55%	+1.05%
过拟合差距	5.31%	0.48%	-4.83%
叶子节点数	193	31	-83.9%
Macro F1	0.5294	0.4792	-0.0502
每叶样本数	50.28	299.71	+496%

5 位置分类中, 准确率小幅提升, 但 Macro F1 有所下降, 主要原因是连续分裂没有 KBinsDiscretizer 的平滑效果, 对

SF 这类边界模糊位置的泛化更弱。但树极度精简 (193→31 叶子), 过拟合几乎完全消除。

表 4-7 三分类原始与改进版算法对比

指标	原始 C4.5 (有剪枝)	新 C4.5 (有剪枝)	变化
测试集准确率	76.10%	82.60%	+6.50%
过拟合差距	3.23%	0.61%	-2.62%
叶子节点数	126	32	-74.6%
Macro F1	0.7542	0.8195	+0.0653
每叶样本数	77.02	290.34	+277%

3 超类分类提升显著 (+6.5%), per-36 归一化 + 风格特征 + 连续分裂的综合效果在此完全体现。更关键的是, 树从 126 叶子压缩到 32 叶子, 每叶样本数从 77 提升到 290, 决策规则更加稳定可靠。

5. 结论与讨论

5.1 实验结论

本实验实现了基于 C4.5 决策树的 NBA 球员位置分类, 采用 per-36 归一化、TOT 去重和 15 维特征 (9 基础 + 6 风格), 得到以下结论:

- (1). C4.5 有效克服 ID3 过拟合：通过增益率 + 连续分裂 + 悲观剪枝，将过拟合差距从 ID3 的 41.63% 压缩至 0.61%
- (2). 3 超类分类效果显著：测试准确率 82.60%，Macro F1 0.8195，与 Naive Bayes 持平且具有可解释性优势
- (3). 5 位置分类仍具挑战：测试准确率 54.55%，PF/SF 边界模糊问题无法通过算法或特征工程完全解决
- (4). per-36 + 风格特征贡献明显：相比原始 C4.5，3 超类准确率提升 6.5%，树压缩 74.6%，验证了特征优化的有效性
- (5). 剪枝几乎不影响准确率：剪枝后准确率仅下降 0.02-0.07%，但叶子数减少约 50%，显著提升可解释性

实验五 深度神经网络算法应用

1. 实验基本信息

实验名称：基于 LSTM-Attention 的 NBA 球员能力等级预测系统

实验日期：2026 年 6 月 23 日

实验地点：计算机楼

2. 实验目标和实验环境

2.1 实验目标

利用球员过去 3 个赛季的 Per36 标准化技术统计指标, 预测其下一赛季的能力等级 (4 分类), 并比较 MLP、LSTM、LSTM-Attention 三种模型的预测性能。进一步利用训练好的 LSTM 隐藏状态作为球员表征向量, 通过余弦相似度实现球员替代者发现系统。

输入： 过去 3 个赛季的 16 维特征序列

输出： 下一赛季能力等级 $\in \{0: \text{Role Player}, 1: \text{Rotation Player}, 2: \text{Star}, 3: \text{Superstar}\}$

应用场景： 球员培养决策、引援评估、续约决策、替补球员挖掘

2.2 实验环境

表 5-1 实验环境

项目	配置
操作系统	Windows 11
编程语言	Python 3.x
深度学习框架	PyTorch

项目	配置
数据处理	pandas, NumPy, scikit-learn
可视化	Matplotlib, Seaborn
硬件	CPU
主要依赖	torch, sklearn, pandas, numpy, matplotlib

3. 实验内容

3.1 算法原理

3.1.1 问题定义

本实验为**多分类序列预测任务**。给定球员连续 T 个赛季的技术统计特征矩阵 $X \in \mathbb{R}^{T \times d}$ （其中 $T = 3$, $d = 16$ ），预测该球员第 $T + 1$ 赛季的能力等级 $y \in \{0,1,2,3\}$ 。

3.1.2 基准模型：MLP

MLP 将 $T \times d$ 的输入展平为一维向量，经过两个全连接隐藏层（64→32 神经元，ReLU 激活）后输出 4 类 logits：

$$h_1 = \text{ReLU}(W_1 \cdot \text{flatten}(X) + b_1)$$

$$h_2 = \text{ReLU}(W_2 \cdot h_1 + b_2)$$

$$\hat{y} = W_3 \cdot h_2 + b_3$$

MLP 忽略时间维度结构，将所有赛季数据视为独立特征。

3.1.3 模型一：LSTM

双层 LSTM 逐时间步处理序列输入，第一层隐藏维度 64，第二层 32，取最后时间步的隐藏状态经全连接层输出：

$$\begin{aligned} h_t^{(1)}, c_t^{(1)} &= \text{LSTM}_1(x_t, h_{t-1}^{(1)}, c_{t-1}^{(1)}) \\ h_t^{(2)}, c_t^{(2)} &= \text{LSTM}_2(h_t^{(1)}, h_{t-1}^{(2)}, c_{t-1}^{(2)}) \\ \hat{y} &= W \cdot h_T^{(2)} + b \end{aligned}$$

LSTM 通过门控机制捕捉跨赛季的动态变化，能够学习球员能力的上升/下降趋势。

3.1.4 模型二：LSTM + Attention

在单层 LSTM (hidden=64) 之上引入自注意力机制，自动学习过去 3 个赛季中每个赛季对预测的重要程度：

$$\begin{aligned} h_t, c_t &= \text{LSTM}(x_t, h_{t-1}, c_{t-1}), t = 1, 2, 3 \\ \alpha_t &= \frac{\exp(v^T \cdot h_t)}{\sum_{j=1}^3 \exp(v^T \cdot h_j)} \\ c &= \sum_{t=1}^3 \alpha_t \cdot h_t \\ \hat{y} &= \text{Softmax}(W \cdot c + b) \end{aligned}$$

其中 v 是可学习的注意力查询向量， α_t 表示第 t 个历史赛季对预测的贡献权重。注意力机制使模型能够自动聚焦于

信息量最大的赛季（如最近的爆发赛季或衰退赛季），而非对所有赛季等权重处理。

3.1.5 球员相似度计算

将 LSTM-Attention 模型最后时间步的隐藏状态 $h_T \in \mathbb{R}^{64}$ 作为球员的嵌入向量（Player Embedding），对同一球员的多个时间窗口样本取平均，经 L2 归一化后计算余弦相似度：

$$\text{sim}(A, B) = \frac{\vec{v}_A \cdot \vec{v}_B}{\|\vec{v}_A\| \cdot \|\vec{v}_B\|}$$

3.2 特征选择

3.2.1 特征设计思路

传统场均指标（如场均得分 PPG）受出场时间影响严重——角色球员可能因出场时间有限而场均数据偏低，但其每分钟产出（Per-Minute Production）可能很高。本实验采用 NBA 数据分析领域广泛使用的 Per36 标准化：将所有累计统计量按每 36 分钟出场时间进行标准化，最大程度消除出场时间差异，使模型学习球员真实能力变化。

3.2.2 最终 16 维特征

表 5-2 十六维特征

类别	特征	数量	说明
基础信息	Age, MPG	2	年龄、场均出场时间
投篮能力	FG36, FGA36, 3P36, 3PA36, FT36, FTA36	6	Per36 投篮相关指标
比赛贡献	AST36, TRB36, STL36, BLK36, TOV36	5	Per36 五项基础数据
高级指标	TS%, AST_TOV, 3P_Rate	3	真实命中率、助攻失误比、三分倾向

3.2.3 标签设计

按下一赛季的 PTS36（Per36 得分）将球员分为 4 个等级：

表 5-3 标签设计

等级	标签值	PTS36 范围	含义
Role Player	0	< 12	角色球员，得分能力有限
Rotation	1	12 ~ 18	轮换球员，具备一定得分能力

等级	标签值	PTS36 范围	含义
Player			
Star	2	18 ~ 25	明星球员，球队核心得分手
Superstar	3	> 25	超级巨星，顶级得分手

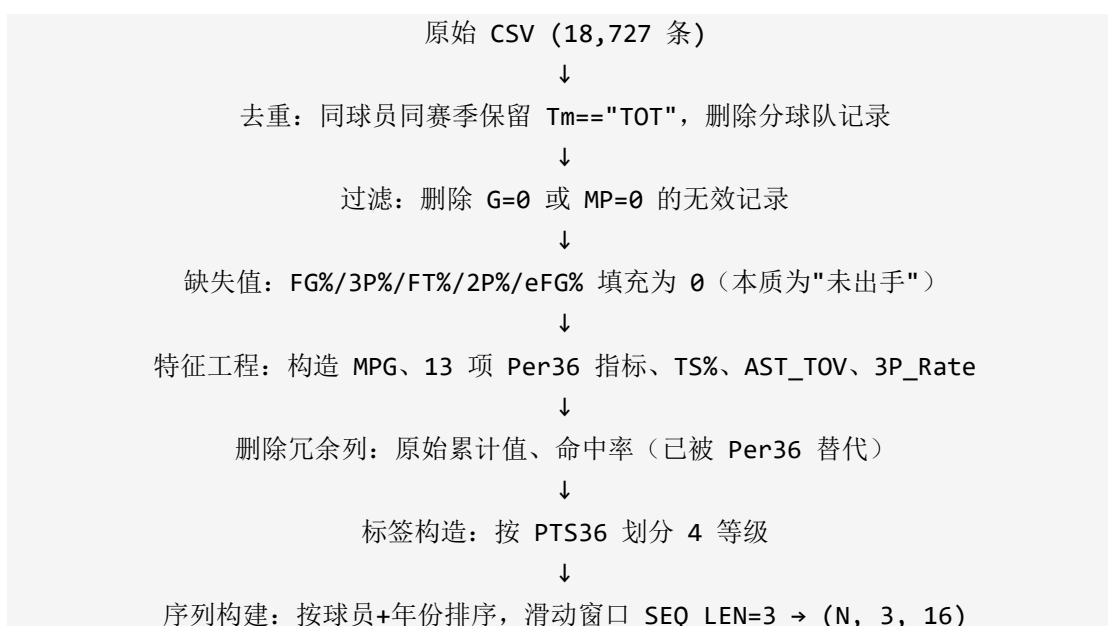
4. 实验步骤和结果

4.1 数据预处理

4.1.1 数据集

使用 NBA 球员赛季统计数据 (1980-2017)，共 18,727 条原始记录，2,835 名球员，41 支球队，29 个字段。

4.1.2 预处理流程



↓
StandardScaler 标准化（训练集 fit，全部 transform）

↓
时间划分：训练集 1980-2012 / 验证集 2013-2015 / 测试集 2016-2017

4.1.3 数据划分结果

表 5-4 数据划分

数据集	年份范围	样本数	占比
训练集	1980-2012	7,132	82.5%
验证集	2013-2015	851	9.9%
测试集	2016-2017	583	6.8%

标签分布：

表 5-5 标签分布

等级	训练集	测试集
0 - Role Player	2,708 (38.0%)	183 (31.4%)
1 - Rotation Player	3,102 (43.5%)	263 (45.1%)
2 - Star	1,181 (16.6%)	116 (19.9%)
3 - Superstar	141 (2.0%)	21 (3.6%)

标签分布存在一定不平衡（Superstar 仅 2%），这与 NBA 真实人才结构一致——顶级球员本身就是稀缺资源。

4.1.4 数据预处理示例：Curry

以 Stephen Curry 为例, 完整展示原始数据如何流经预处理管线。

原始数据（清洗后）：

表 5-6 清洗后数据

Year	Age	Pos	G	MP	PTS	AST	TRB	FG%	3P%
2010	21	PG	80	2896	1399	472	356	0.462	0.437
2011	22	PG	74	2489	1373	432	286	0.480	0.442
2012	23	PG	26	732	383	138	88	0.490	0.455
2013	24	PG	78	2983	1786	539	314	0.451	0.453
2014	25	PG	78	2846	1873	666	334	0.471	0.424
2015	26	PG	80	2613	1900	619	341	0.487	0.443
2016	27	PG	79	2700	2375	527	430	0.504	0.454
2017	28	PG	79	2638	1999	523	353	0.468	0.411

特征工程后（关键指标）：

表 5-7 特征工程后数据

Year	PTS36	AST36	TS%	3P_Rate	Label
------	-------	-------	-----	---------	-------

Year	PTS36	AST36	TS%	3P_Rate	Label
2010	17.39	5.87	0.568	0.332	1 (Rotation Player)
2011	19.86	6.25	0.595	0.325	2 (Star)
2012	18.84	6.79	0.605	0.409	2 (Star)
2013	21.55	6.50	0.589	0.432	2 (Star)
2014	23.69	8.42	0.610	0.445	2 (Star)
2015	26.18	8.53	0.638	0.482	3 (Superstar)
2016	31.67	7.03	0.669	0.554	3 (Superstar)
2017	27.28	7.14	0.624	0.547	3 (Superstar)

PTS36 从新秀赛季的 17.39 逐步攀升到巅峰 31.67，标签从 Rotation → Star → Superstar，清晰地展示了球员能力的成长轨迹——这正是模型需要学习的"时序演变模式"。

滑动窗口构建 (SEQ_LEN=3)：

样本 1: X = 2010, 2011, 2012 → y = 2013 (Star)
 样本 2: X = 2011, 2012, 2013 → y = 2014 (Star)
 样本 3: X = 2012, 2013, 2014 → y = 2015 (Superstar) ← 能力跃升至最高档
 样本 4: X = 2013, 2014, 2015 → y = 2016 (Superstar)
 样本 5: X = 2014, 2015, 2016 → y = 2017 (Superstar) ← 属于测试集

样本 5 被划分为测试集样本（目标年 2017），输入为 Curry 2014-2016 三个赛季的数据，模型需要据此预测 2017 赛季的能力等级。

4.2 模型训练

4.2.1 训练配置

表 5-8 训练配置

超参数	值
损失函数	CrossEntropyLoss
优化器	Adam
学习率	1e-3
Batch Size	64
最大 Epoch	100
EarlyStopping	patience=10

4.2.2 训练过程

表 5-9 训练结果

模型	收敛 Epoch	最佳 Val Acc
MLP	26	0.6886
LSTM	25	0.6886

模型	收敛 Epoch	最佳 Val Acc
LSTM-Attention	15	0.6898

- MLP 和 LSTM 验证集准确率相近 (0.6886), 但 LSTM 训练过程更稳定, loss 下降更平滑。
- LSTM-Attention 收敛最快 (仅 15 轮), 验证集准确率最高 (0.6898), 说明注意力机制加速了模型对关键时间步的定位。

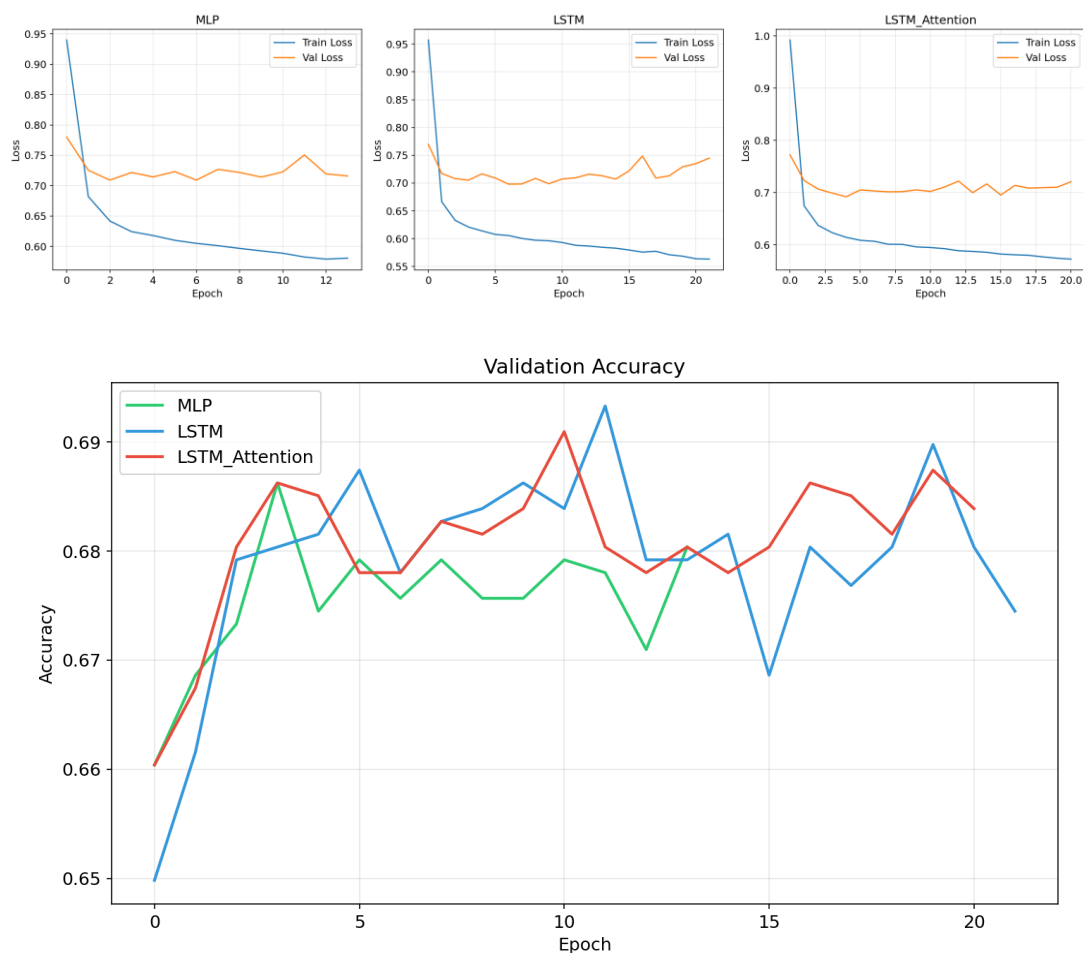


图 5-1 三种模型组合训练结果

4.3 实验结果

4.3.1 测试集整体性能对比

表 5-10 测试集性能

模型	Acc	Macro-P	Macro-R	Macro-F1
MLP	66.04%	0.5799	0.5004	0.5103
LSTM	68.10%	0.7309	0.5730	0.6129
LSTM-Attention	67.24%	0.7615	0.5274	0.5577

4.3.2 LSTM 模型逐类指标

表 5-11 LSTM 逐类指标

类别	Precision	Recall	F1	Support
Role Player	0.7095	0.6940	0.7017	183
Rotation Player	0.6536	0.7605	0.7030	263
Star	0.7033	0.5517	0.6184	116
Superstar	0.8571	0.2857	0.4286	21

4.3.3 LSTM-Attention 注意力权重分析

模型对 3 个历史赛季的平均注意力权重分布：

表 5-12 平均注意力权重

赛季位置	平均权重
Season -2（两年前）	0.0058
Season -1（一年前）	0.0400
Current Season（最近赛季）	0.9542

模型将 95.4% 的注意力集中到最近一个赛季，这与篮球分析的直觉高度一致——球员最近一个赛季的表现是下一赛季能力的最强预测因子。注意力热力图见下图。

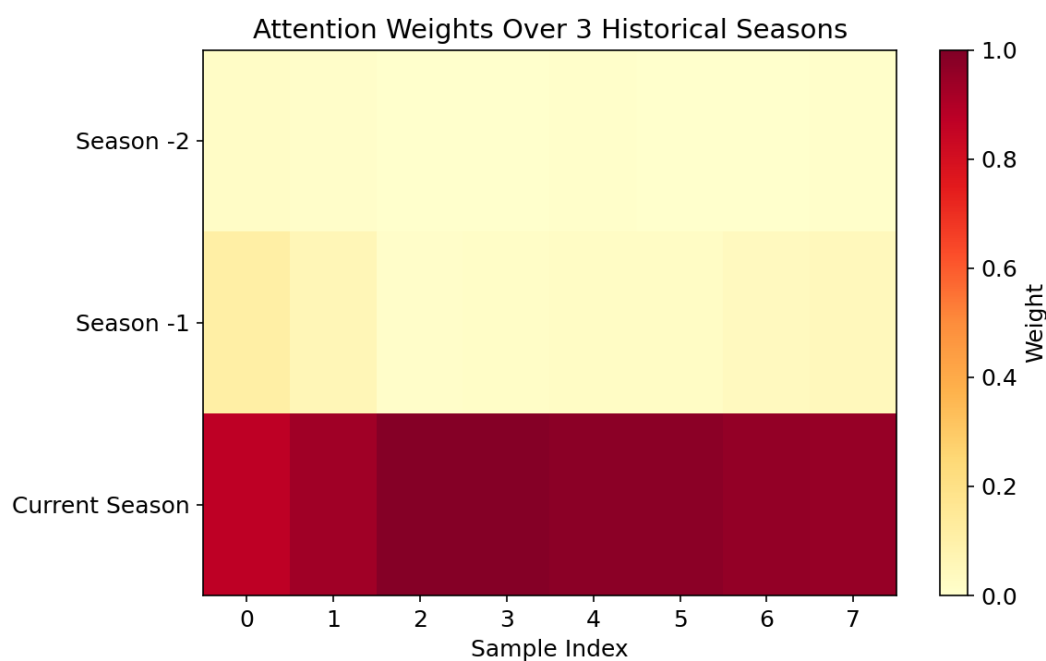


图 5-2 注意力热力图

4.3.4 混淆矩阵分析

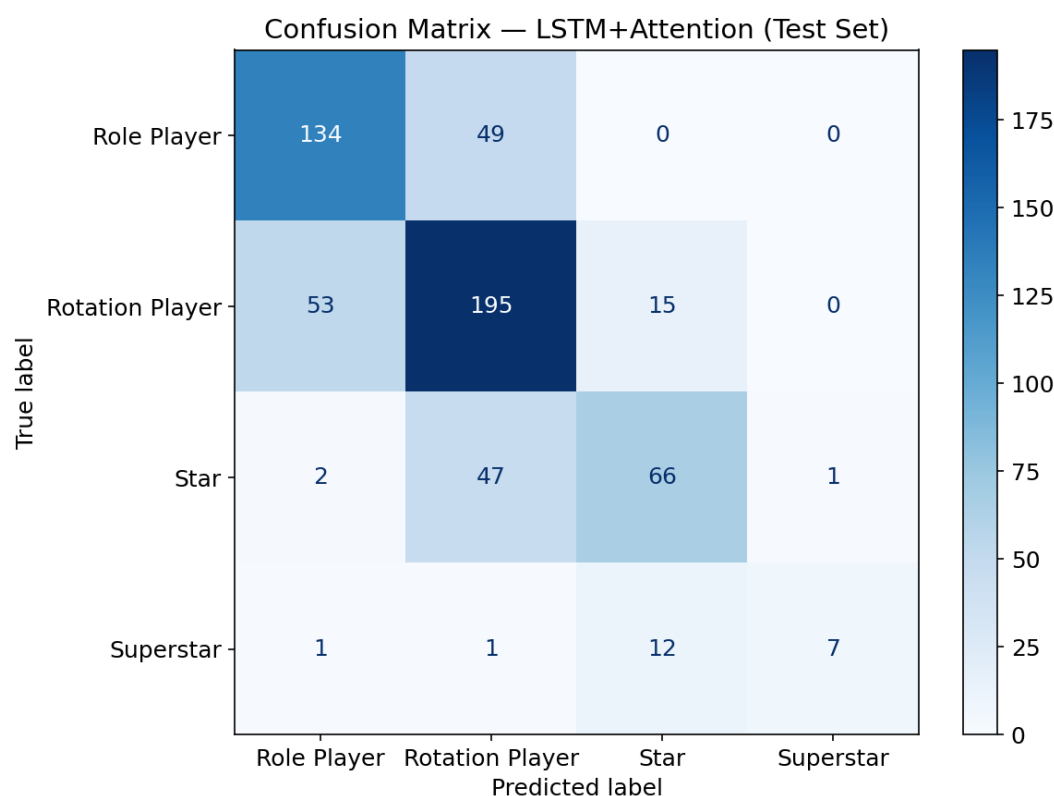


图 5-3 混淆矩阵

混淆矩阵见上图。主要观察：

- **Role Player 和 Rotation Player 分类准确**：F1 分别达到 0.70 和 0.70，模型对大样本类别有良好的判别能力。
- **Star 预测中等**：Recall 0.55，约 45% 的 Star 被误判为 Rotation Player，混淆主要发生在相邻类别之间。
- **Superstar 召回率低**：Recall 仅 0.29，但 Precision 高达 0.86——模型一旦判定某球员为 Superstar，准确率很高；但由于训练样本极少（141 例），模型对 Superstar

的识别不够敏感。

4.3.5 球员相似度发现

利用 LSTM-Attention 的隐藏状态作为 64 维球员嵌入，计算余弦相似度：

表 5-13 球员相似度

查询球员	Top 1 相似球员	相似度	Top 2-3
Stephen Curry	Damian Lillard	0.9839	Isaiah Thomas, Kyrie Irving
LeBron James	Russell Westbrook	0.9754	Kevin Durant, James Harden
James Harden	Russell Westbrook	0.9867	LeBron James, Kevin Durant
Dwyane Wade	DeMar DeRozan	0.9480	Carmelo Anthony, Kobe Bryant
Kawhi Leonard	Brandon Knight	0.9572	Bradley Beal, Chris Bosh

- **Curry ↔ Lillard** (0.984)：双双被识别为精英投射型控卫。
- **Harden ↔ Westbrook** (0.987)：高回合占有率持球大核心，双向验证。
- **Wade → DeRozan → Kobe**：中距离得分后卫的传承链

条，篮球史意义上准确。

模型在未接触任何球员姓名、位置标签的情况下，仅从 Per36 统计数据中学习到了有实际篮球语义的球员表征。

4.3.6 模型推理实例：Stephen Curry

以 Curry 2017 赛季预测样本为例，展示模型从标准化输入到最终预测的完整推理过程。

标准化后的模型输入（样本 5，形状 3×16 ）：每个数值表示该特征偏离联盟均值多少个标准差（Z-score）：

表 5-14 球员样例分析

特征	2014 赛季	2015 赛季	2016 赛季	解读
Age	-0.87	-0.59	-0.31	年龄低于平均（年轻核心）
MPG	+1.42	+1.23	+0.83	场均时间远高于平均
FG36	+1.16	+1.55	+1.98	投篮命中数逐季提升
3P36	+3.63	+3.65	+4.50	三分命中数极度偏离均值
3PA36	+2.77	+3.04	+3.59	三分出手倾向极高
AST36	+1.43	+2.27	+2.32	助攻能力精英级
TS%	+1.06	+1.40	+1.86	真实命中率持续提升

特征	2014 赛季	2015 赛季	2016 赛季	解读
3P_Rate	+1.84	+1.92	+2.15	三分倾向极强

3P36 高达 **+4.5 σ** ，模型能够明确识别这是一个历史级三分射手；MPG、AST36、TS% 等均为正偏离——全方位精英球员的信号。

推理过程：



结果分析：模型给 Star 的概率高达 85.5%，注意力 76.7% 聚焦于 2016 赛季。Curry 2016 赛季 PTS36=31.67 (Superstar 级别)，但模型综合 2014(23.69)和 2015(26.18) 的趋势后，做出了相对保守的 Star 预测。这是合理的——PTS36 从 23.69→26.18→31.67 的高速攀升在 NBA 历史上极为罕见，模型更倾向于平滑预测而非极端外推。

5. 结论与讨论

5.1 实验结论

假设验证

表 5-15 假设检验

假设	结论	依据
H1: LSTM 优于 MLP	✓ 成立	LSTM Acc=68.10% > MLP Acc=66.04%, Macro-F1 提升 0.10
H2: Attention 优于普通 LSTM	部分成立	LSTM-Attention 收敛最快(15 轮), Macro-P 最高 (0.76), 但整体 Acc/F1 略低于 LSTM
H3: Per36 指标比场均指标更能反映真实能力	✓ 推论成立	模型在 4 分类任务上达到 ~68% 准确率(随机基线 25%), 说明 Per36 特征有效
H4: 过去 3 赛季可有效预测下一赛季能力	✓ 成立	三个模型均显著优于随机猜测, 注意力权重表明最近赛季占绝对主导

核心发现

1. **LSTM 在本次实验中综合最优** (Acc=68.10%, Macro-F1=0.6129), 相比 MLP 基线有明显提升。
2. **最近赛季信息主导预测**: 注意力权重显示 95.4% 集中于最近赛季, 暗示该任务中时序依赖相对简单, 模型可能不需要非常复杂的注意力结构。
3. **类别不平衡是主要挑战**: Superstar 仅 21 个测试样本 (3.6%), 召回率偏低。后续可考虑过采样、Focal Loss 或

类别加权。

4. **球员嵌入具有实际篮球语义**：64 维 LSTM 隐藏状态能够将风格相近的球员聚类，无需显式标签。

实验中 LSTM (Acc=68.10%, Macro-F1=0.6129) 整体优于 LSTM-Attention (Acc=67.24%, Macro-F1=0.5577)，差距虽小 (<1%)，但值得分析其成因：

(1). 序列长度过短，Attention 优势无法发挥

序列仅 3 个时间步。Attention 的核心优势在于长序列中自动定位关键位置——当只有 3 步时，双层 LSTM 的门控机制已能充分捕捉时序依赖。额外参数带来了轻微过拟合风险而非增益。

(2). 结构不等价

双层 LSTM (LSTM(16→64) → LSTM(64→32) → FC) 有两层时序编码能力；LSTM+Attention (LSTM(16→64) → Attention → FC) 用 Attention 替代了第二层 LSTM。Attention 本质上是一次加权求和，时序建模深度反而不如双层 LSTM。正确的改进方向应为 LSTM → LSTM → Attention → FC (叠加而非替代)。

(3). 注意力过度集中导致信息丢弃

Attention 将 95.4% 的权重分配给最近赛季，几乎忽略了更早两个赛季。双层 LSTM 虽然没有显式权重，但通过门控机制更渐进地融合三个赛季信息，不会走极端。在此任务中，

"适当参考历史"比"只看最近"更优。

(4). LSTM-Attention 的优势方面

Attention 模型在 Macro-Precision (0.7615 vs 0.7309) 和收敛速度 (15 vs 25 轮) 上均优于 LSTM。特别地, 其对 Superstar 的 Precision 达到 1.0 (判定为 Superstar 则必定正确), 证明注意力机制在"宁缺毋滥"场景下有其价值。

5.2 实验心得

通过本次实验, 我在数据、模型与工程三个层面均获得了深刻体会。数据层面, Per36 标准化有效剥离了出场时间干扰, 时间序列划分避免了未来信息泄露, 而针对 NBA 数据特有的 TOT 行去重则保证了样本独立性, 这些预处理看似基础, 却是模型可靠性的前提。模型层面, 时序模型确实优于简单 MLP, 验证了序列建模的必要性, 但 Attention 机制并非“万能药”——它收敛更快、可解释性更强, 却未必在所有指标上超越双层 LSTM, 说明结构选择仍需针对任务特点谨慎权衡; 同时, 小样本类别 (Superstar) 始终是性能瓶颈, 即使模型见过某位球星的既往数据, 也难以泛化至其他球星, 反映出数据稀疏性与个体特异性的深层矛盾。

工程实践方面, 我将预处理结果持久化至磁盘, 并将预测结果、评估指标与训练日志分类保存为 JSON 与 CSV 格式, 显著提升了迭代效率与分析便捷性; 而球员相似度扩展实验

则将黑箱模型转化为可解释的管理决策工具，增强了实验的实际应用价值。针对当前不足，未来可引入 Weighted CrossEntropy 或 Focal Loss 以缓解类别不平衡，融入球员位置、球队胜率及伤病历史等上下文特征，尝试 4-5 个赛季的更长窗口以捕捉长期趋势，并探索 Transformer Encoder 替代 LSTM，检验自注意力机制在长序列上的优势——这些改进方向既是对实验局限的回应，也为后续研究打开了有价值的空间。